New Limits for Homomorphic Encryption

Sven Schäge *, Marc Vorstermans
Eindhoven University of Technology, the Netherlands
s.schage@tue.nl, m.h.l.vorstermans@tue.nl

Abstract

We make progress on the foundational problem of determining the strongest security notion achievable by homomorphic encryption. Our results are negative. We prove that a wide class of semi-homomorphic public key encryption schemes (SHPKE) cannot be proven IND-PCA secure (indistinguishability against plaintext checkability attacks), a relaxation of IND-CCA security. This class includes widely used and versatile schemes like ElGamal PKE, Paillier PKE, and the linear encryption system by Boneh, Boyen, and Shacham (Crypto'04). Besides their homomorphic properties, these schemes have in common that they all provide efficient algorithms for recognizing valid ciphertexts (and public keys) from public values. In contrast to CCA security, where the adversary is given access to a decryption oracle, in a PCA attack, the adversary solely has access to an oracle that decides for a given ciphertext/plaintext pair (c, m) if decrypting c indeed gives m. Since the notion of IND-PCA security is weaker than IND-CCA security, it is not only easier to achieve, leading to potentially more efficient schemes in practice, but it also side-steps existing impossibility results that rule out the IND-CCA security. To rule out IND-PCA security we thus have to rely on novel techniques. We provide two results, depending on whether the attacker is allowed to query the PCA oracle after it has received the challenge (IND-PCA2 security) or not (IND-PCA1 security – the more challenging scenario). First, we show that IND-PCA2 security can be ruled out unconditionally if the number of challenges is smaller than the number of queries made after the challenge. Next, we prove that no Turing reduction can reduce the IND-PCA1 security of SHPKE schemes with $O(\kappa^3)$ PCA queries overall to interactive complexity assumptions that support t-time access to their challenger with $t = O(\kappa)$. To obtain our second impossibility results, we develop a new meta-reduction-based methodology that can be used to tackle security notions where the attacker is granted access to a *decision* oracle. This makes it challenging to utilize the techniques of existing meta-reduction-based impossibility results that focus on definitions where the attacker is allowed to access an inversion oracle (e.g. long-term key corruptions or a signature oracle). To obtain our results, we have to overcome several technical challenges that are entirely novel to the setting of public key encryption.

Keywords: Impossibility, Homomorphic Encryption, IND-PCA, ElGamal Encryption, Paillier Encryption, Meta-Reduction

*Sven Schäge was supported by the Smart Networks and Services Joint Undertaking (SNS JU) under the European Union's Horizon Europe research and innovation programme in the scope of the CONFIDENTIAL6G project under Grant Agreement 10109643. The contents of this publication are the sole responsibility of the authors and do not in any way reflect the views of the EU.

1 Introduction

Fully homomorphic encryption (FHE) is a very powerful tool that allows for conceptually elegant solutions to many challenging problems in cryptography. When using FHE in practice to design secure systems, knowing exactly what security guarantees the FHE scheme provides is of the utmost importance. Consequently, the problem of narrowing down the strongest achievable security notion for FHE has gained much attention in the last two decades [32, 48, 36, 2, 7, 19, 35, 46], leading to both positive and negative results including proposals of new security notions on the one hand and novel attacks and impossibility results on the other hand. When we switch to the more restricted semi-homomorphic public key encryption (SHPKE) systems that only support a single homomorphic operation on encrypted plaintexts, the problem typically becomes even harder: any attack on an SHPKE scheme where the attacker only exploits one homomorphic operation on encrypted messages immediately works a forteriori if the attacker is additionally given access to a second operation. Moreover, coming up with attacks on SHPKE schemes in the first place is more challenging, precisely because the attacker only has access to a single homomorphic operation.

In cryptography, SHPKE systems like ElGamal public-key encryption (PKE) or Paillier PKE have diverse application scenarios and are widely used. Most recently, they have gained renewed attention in the realm of threshold cryptography, where their homomorphic properties allow them to combine different key shares efficiently with each other, e.g. [23, 33]. Existing SHPKE schemes are typically accompanied by proofs of IND-CPA security under non-interactive security assumptions like the Decisional Diffie-Hellman assumption or the Decisional Composite Residuosity assumption. Moreover, it is well-known that they cannot achieve indistinguishability under chosen ciphertext attacks (IND-CCA2 security) because of the malleability of their ciphertexts.

The problem of whether these schemes provide IND-CCA1 (or lunchtime) security[1] – a slightly weaker notion where the attacker is not allowed to query the decryption oracle after the challenge query has been issued – has only recently been answered negatively [48]. It remains an important foundational problem to investigate the possibility to achieve weaker security notions for these widespread SHPKE schemes that lie somewhere between IND-CCA2/IND-CCA1 security and IND-CPA security. One well-known notion that can be found here is the definition of indistinguishability under plaintext checkable attacks (IND-PCA) a natural relaxation of IND-CCA security. In contrast to IND-CCA security, it only grants the attacker access to a *decision oracle* in the learning phase, that on input ciphertext/plaintext pair (c, m) outputs a validity bit $w \in \{0, 1\}$ indicating whether decrypting c indeed gives m. Since the attacker is only allowed to query the decisional PCA oracle as opposed to having access to an inversion (decryption) oracle in the IND-CCA security game, IND-PCA security is a provably weaker notion. This has two important consequences.

First, IND-PCA security can generally be achieved more easily in constructions than IND-CCA security. As a result, IND-PCA secure PKE schemes can generally be more efficient. Moreover, since the security notion is less demanding, they may exhibit desirable features that are useful in more complex systems, like the IND-PCA secure variant of Cramer-Shoup encryption in [1] that is compatible with Groth-Sahai proofs. At the same time, despite their weaker security properties, IND-PCA secure schemes can nevertheless be very useful in practice. Remarkably, in many important PKE-based constructions, IND-PCA security is in fact sufficient. For example, as observed by [1], all the password-based security protocols in [24, 25, 29] that are built from IND-CCA secure PKE schemes can actually safely rely on IND-PCA secure PKE schemes.

Second, since IND-PCA is provably weaker than IND-CCA security, existing impossibility

[1]The proof actually showed the stronger result that SHPKE schemes cannot be proven OW-CCA1 secure, a notion formally incomparable to IND-PCA security.

results that rule out IND-CCA security do not apply. Despite their importance in practice, until now it is thus simply unknown whether semi-homomorphic encryption schemes can achieve IND-PCA security at all.

In light of this, we investigate the following question.

Can Semi-Homomorphic PKE be shown IND-PCA secure?

We answer the above question negatively.

Setting the Stage: A Weaker Security Notion. To obtain our results, we start by introducing a security game that we call *weak* indistinguishability against plaintext checkable attacks (wIND-PCA). In contrast to the classical notion of IND-PCA security, the attacker is *not* allowed to send a plaintext to the challenger that may be used to compute the challenge ciphertext c^*. Instead, c^* is *always* computed by encrypting a uniformly random message m_1^* that has been drawn by *the challenger*. Now, c^* is sent back to the attacker along with the real message m_1^* (real case) or an independent random message m_0^* (random case). The goal of the attacker is to distinguish the real from the random case. Since the attacker has strictly less freedom than in the IND-PCA game, the security notion for wIND-PCA is weaker and any impossibility result for wIND-PCA security immediately holds for IND-PCA security as well. Besides leading to more general impossibility results, focusing on the notion of wIND-PCA security provides a conceptual advantage: both the queries sent in the learning phase and the challenge itself consist of ciphertext/plaintext pairs. All our impossibility results consider, in fact, wIND-PCA security and thus provide a more general result that a fortiori rules out IND-PCA security.

Agnostic Randomization with Strong Hiding. Next, we introduce a specific randomization algorithm $\mathsf{Randomize}$ that, as we show, every SHPKE admits. This algorithm randomizes ciphertext/plaintext pairs $s = (c, m)$ to other pairs $s' = (c', m')$ with the crucial restriction that the validity bit remains untouched. This means that s' is a valid ciphertext/plaintext pair if and only if s is valid. Remarkably, under this condition, the randomization is perfect (except with statistically low probability) and *agnostic* to the actual validity bit of the input s. Accordingly, if s is not a valid pair, s' will be distributed *uniformly* in the set INV of all invalid ciphertext/plaintext pairs, even for a distinguisher who knows the original $s = (c, m)$. Likewise, if s is valid, then s' will be distributed uniformly in the set VAL of valid ciphertext pairs.[2] This provides statistically strong hiding properties since any two input pairs $s, s' \in \mathsf{VAL}$ (respectively $s, s' \in \mathsf{INV}$) have the same the output distribution when applying $\mathsf{Randomize}$. It is highly useful that the algorithm $\mathsf{Randomize}$ does not need the validity bit of s as input.

Two Results. Next, we consider two different scenarios. In the first, the attacker makes queries to the PCA oracle *after* it has received the challenges $s_1^* = (c_1^*, m_1^*), \ldots, s_p^* = (c_p^*, m_p^*)$. Making a similar distinction as between the subnotions for IND-CCA security, we refer to the resulting game as wIND-PCA2 security. In contrast, wIND-PCA1 security only considers attackers that query the oracle *before* the attacker receives the challenges. As our first result, we unconditionally rule out that SHPKE schemes achieve wIND-PCA2 security. The idea, heavily inspired by the well-known impossibility result for IND-CCA2 security, is to exploit the homomorphic properties of ciphertexts in the SHPKE scheme and randomize s^* to obtain a new pair $s'^* = (c'^*, m'^*)$ that is queried to the PCA oracle. Since $\mathsf{Randomize}$ leaves the validity bit unchanged, i.e. $w'^* = w^*$, a valid response to s' will immediately transfer to a valid response for s^*. Thus, the attacker can

[2] We assume that $\mathsf{VAL} \cap \mathsf{INV} = \varnothing$.

always win in the wIND-PCA2 security game. We believe that the impossibility of IND-PCA2 security can be shown for schemes with less demanding properties. In particular, for the result to hold, we do not need randomization algorithms with strong hiding properties, something which is required in the IND-PCA1 impossibility result.

wIND-PCA1 Security. Settling the question of whether SHPKE schemes are wIND-PCA1 secure is much more complicated. To answer it, we follow the meta-reduction methodology introduced by Boneh and Venkatesan [8]. The central idea is to describe an ideal successful attacker A that can be efficiently simulated by a meta-reduction M towards the reduction B. Meta-reduction M rewinds B to simulate the ideal attacker. As a consequence, the combination of M and B can be used to break the security assumption that the reduction reduces to. This approach yields strong impossibility results that do not idealize mathematical structures in contrast to, for example, the generic group model [50, 37] or the generic ring model [30]. The only restriction we make is that the reduction uses inefficient attackers in a black-box way. Intuitively, our idea to efficiently simulate the ideal attacker is to exploit that the meta-reduction can rewind the reduction. Similar to before, the goal of M is to randomize the challenge s and send the result s' as a query in the learning phase. However, to make sure that the reduction accepts this rewound query, we not only have to make sure that the query is distributed like any other query in the learning phase. Since general reductions can rewind the attacker, we also have to make sure that the entire *behavior* of the simulated attacker M is indistinguishable from that of A, even if B can rewind the attacker and run several attacker instances concurrently. To do this, we have to overcome several obstacles that we describe in more detail shortly. Our main result shows the impossibility of general reductions from the wIND-PCA1 game with $3\kappa(2t + 2\kappa + 2u)$ PCA queries to any t-interactive complexity assumption (tICA). Here κ is the security parameter and u is the maximum number of attacker instances that may be run by B. In Appendix B, we also provide an impossibility result with quantitatively better bounds that, however, only holds for simple reductions. In contrast to general Turing reductions, simple reductions only execute the attacker once and without rewinding.[3] Let us now describe several challenges that we have to overcome to obtain our impossibility result for wIND-PCA1 security.

Rewound Queries Must not Disturb External ICA Communication. After our meta-reduction receives the challenge statement $s^* = (c^*, m^*)$, it rewinds the reduction back to the learning phase with the aim of exploiting the reduction itself to provide the answer to s^*. To ensure that the meta-reduction cannot immediately be distinguished from the ideal attacker, s^* needs to be randomized to look like a new ciphertext/plaintext pair. Moreover, it is highly important that the additional query sent to the reduction after the rewinding will, with non-negligible probability, not disturb B's overall interaction with the interactive complexity assumption. In particular, we have to make sure that this rewound query will not make B query its ICA-challenger at all to avoid any influence of the rewinding on the overall number of queries made to the ICA-challenger. This is important to guarantee that ideal attacker and meta-reduction remain indistinguishable from the viewpoint of the reduction.

Dealing with Correlated Validity Bits. It turns out, that solving this problem is much harder than in previous results like [48] where the challenge s^* and the randomized statement s'^* are statistically independent. In our setting, a major problem is that, even though $s'^* = (c'^*, m'^*)$ is a 'good' randomization of $s^* = (c^*, m^*)$, the two values have the same validity bit and thus

[3] Although simple reductions are widespread in cryptography, focussing on them restricts the freedom of the reduction considerably. Moreover, there are important classes of reductions that are not simple like classical proofs of knowledge in zero-knowledge protocols that rely on rewinding.

are highly correlated. Any reduction that recognizes that the *validity bit of both statements is equal* can thus base its behavior in some subtle way on this fact. This is problematic as the reduction may specifically choose a strategy where external queries to the ICA-challenger are, for example, *always* made by the reduction if the queried pair has validity bit $w = 0$. If the reduction now produces challenge (c^*, m^*) with validity bit $w^* = 0$ as well, the meta-reduction cannot successfully use a naive rewinding approach because randomizing (c^*, m^*) to (c'^*, m'^*) will leave the validity bit unchanged and querying (c'^*, m'^*) to B will *always* disturb the interaction between B and the interactive complexity challenge. If the meta-reduction rewinds the reduction and sends $s'^* = (c'^*, m'^*)$ in the learning phase, reduction B will thus always invoke an external query and *never* respond to the query s'^* without using an additional external query.

Statistical Solution for Correlated Validity Bits and Multiple Attacker Instances. We solve this problem statistically. To this end, we describe an algorithm $\mathsf{RSample}$ that can be used by the ideal attacker to draw the corresponding validity bits w in its queries *uniformly at random* although the sets INV and VAL can have vastly different sizes. Half of the overall y queries that the attacker makes thus have $w = 0$ (or 1) on average. Hoeffding's inequality assures that with overwhelming probability the actual number of validity bits that are equal to 0 (or 1) is very close to the average $y/2$. Luckily and similar to $\mathsf{Randomize}$, we are able to show that every SHPKE encryption scheme in fact gives rise to such an algorithm $\mathsf{RSample}$. Intuitively, we use this to guarantee that the probability of having more than $t + u$ and fewer than $y - (t + u)$ queries in the learning phase (before rewinding) be equal to 0 is overwhelming. (Symmetrically, the probability that at least $t+u$ and at most $y-(t+u)$ values equal 1 is overwhelming.) We set up the parameters such that there is always one query, a so-called useful rewinding spot, with the property that when the reduction B delivers the response to the query i) B has not made a new query to its ICA-challenger (that only accepts t queries in total), and ii) B has not requested any of the other $u - 1$ attacker instances to output an answer to their challenge. While the first condition effectively guarantees that the interaction of the reduction with its tICA challenger does not get disturbed, the second condition guarantees that the runtime of the meta-reduction does not blow up exponentially. The underlying problem is well-known in the realm of zero-knowledge proofs [47, 9, 44, 15, 10]. In a nutshell, the challenge is that the reduction could wait until all other attacker instances have successfully delivered solutions to their challenges before it finally outputs its response to the rewound query. Since each of these solutions can only be obtained by the meta-reduction via rewinding, the rewinding success of the u attacker instances could depend on each other in a way that increases the runtime exponentially. However, once a useful rewinding spot exists, the meta-reduction M can, with non-negligible success probability, guess when it occurs and send the rewound query. The response w'^* to s'^* is also a solution w^* to the challenge s^*. Validity bit w^* will be sent back to the reduction B and, by assumption, the reduction breaks the tICA. Since B and M are efficient, this would break the assumed security of the tICA, which ultimately shows that B cannot exist.

Dealing with Rewinding Reductions and Incorrect Responses. Another central problem that we have to overcome when dealing with general Turing reductions is that a naïve meta-reduction is not able to verify the responses of the reduction [48] *after rewinding*. Before rewinding, in contrast, both the ideal attacker and meta-reduction can easily verify the responses of the reduction if they produce the queries that they sent to B themselves and thus know their validity bits. However, in the rewinding phase, the meta-reduction is *not* able to verify the responses of the reduction as it does not know the validity bit of the challenge statement. (This is what the meta-reduction aims to reveal in the first place.) This is problematic, for example when faced with a rewinding reduction that always responds to any query with an incorrect

answer first. Observe that such a reduction is still efficient: if the attacker aborts because of an incorrect response, the reduction can simply rewind the attacker and send a correct response instead. The meta-reduction's inability to recognize incorrect responses after rewinding could now be used by the reduction to tell the ideal attacker and meta-reduction apart, because the ideal attacker (that does not use rewinding) always knows the correct response and thus may always abort if it is given an incorrect response.

Solution: Adding Redundancy. To overcome this problem, we implement an additional mechanism such that the meta-reduction can recognize if the response of the reduction is incorrect – *even if it does not know the validity bit of the challenge*. To achieve this in our setting, we develop entirely novel techniques. Technically we proceed as follows. First, we consider a security game that is even weaker than wIND-PCA security where queries are vectors, that consist of 3κ pairs $(c_1, m_1), \ldots, (c_{3\kappa}, m_{3\kappa})$. This game keeps the overall number of statements of the wIND-PCA game, such that the resulting game is simply less adaptive than the wIND-PCA1 game and thus provably weaker. The vectors are carefully constructed as follows: first, we draw a random bit w. Next, we construct three random pairs $s_1 = (c_1, m_1)$, $s_2 = (c_2, m_2)$, and $s_3 = (c_3, m_3)$ conditioned on the fact that s_1 has validity bit 0, s_2 has validity bit 1, and s_3 has validity bit w. Finally, we randomize each s_i a total of κ times to obtain 3κ new pairs in total. As a last step, we permute these 3κ pairs with a secret random permutation to obtain $(c_1, m_1), \ldots, (c_{3\kappa}, m_{3\kappa})$. The reduction will now have to provide as a response all the 3κ validity bits in the correct order (exactly 2/3 of them have value w). The ideal attacker knows, as well as the meta-reduction *before* it rewinds B, the validity bit of each ciphertext/plaintext pair in the vector. In the rewinding phase, the meta-reduction essentially sets $s_3 = (c^*, m^*)$ to be equal to the challenge statement such that w is determined by the validity bit of the challenge $s^* = (c^*, m^*)$. However, this validity bit is unknown to the meta-reduction. We show, however, that the only way for the reduction to successfully output an incorrect response now is by flipping *all* the validity bits at the positions with randomizations of s_3. In all other cases, the meta-reduction will notice that the response is incorrect: for 2/3 of the components the meta-reduction knows what the validity bit is and where it should occur (those based on s_1 and s_2). Only for the validity bits derived from s_3 the meta-reduction does not know what the validity bit is. However, it knows where these derived components will be and that they must all have the same validity bit. Moreover, due to the hiding properties of $\mathsf{Randomize}$ the exact positions that belong to s_3 are *perfectly* hidden in a subvector of size 2κ. This subvector consists of all pairs that have the validity bit that is more frequent overall. To only flip the validity bits that correspond to s_3, the best the reduction can do is thus correctly guess which κ of the 2κ pairs with validity bit w belong to s_3 since the random permutation is kept secret. The probability for this is $\binom{2\kappa}{\kappa}^{-1}$. which is statistically small. We obtain the following theorem.

Theorem 2 (Second Main Result, Informal). *Let* PKE *be an SHPKE with security parameter* κ*. There exists no PPT reduction* B *that, while creating at most* u *attacker instances, reduces the security of the wIND-PCA1 security game with* $3\kappa(2t + 2\kappa + 2u)$ *adaptive PCA queries to the security of any* t*-interactive complexity assumption* ICA*.*

Related Work. We add new results to the line of work in [17, 16, 41, 42, 22, 49, 4, 20, 21] that focuses on the (in)feasibility of cryptographic primitives. In contrast to previous meta-reduction-based works like [11, 28, 27, 31, 3, 38, 48] that tackle inversion-type security notions, we concentrate on the *decisional* problem of recognizing valid ciphertext/plaintext pairs. In this way, our work complements the recent results of [48] that focuses on the computational problem of computing plaintexts from ciphertexts. However, there are major differences in the

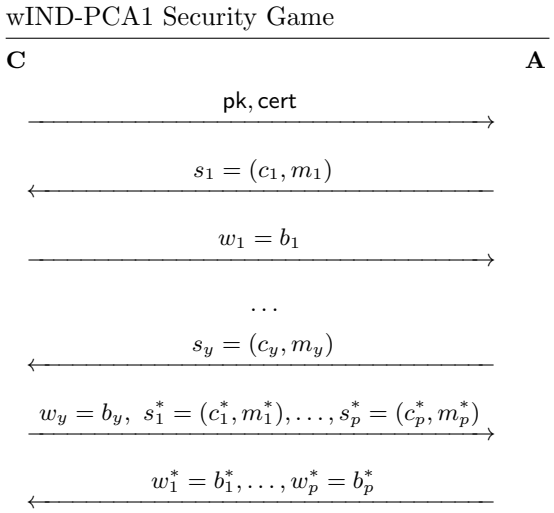

Figure 1: The wIND-PCA1 security game between challenger **C** and attacker **A**. On initialization, a public key pk together with a certificate cert of an SHPKE are sent to the attacker. The attacker may query y PCA queries of the form $s_i = (c_i, m_i)$ for $i \in [y]$. The challenger will respond with $w_i = 1$ if s_i is a valid ciphertext/plaintext pair for the SHPKE. Otherwise, **C** responds with bit $w_i = 0$. After t queries, the attacker is given p challenge statements $s_1^* = (c_1^*, m_1^*), \ldots, s_p^* = (c_p^*, m_p^*)$, for which it has to determine the corresponding validity bits $w_1^*, \ldots, w_p^*$.

two settings, and we have to develop novel solutions to new problems: in our setting we do not need to consider the re-randomizability of witnesses since the witness, i.e. the validity bit, is unique.[4] In this sense, our work is closer to earlier works on meta-reductions that entirely rely on unique witnesses like [11, 28]. Moreover, our randomization technique is very different from the one used in [48]. Whereas their randomization produces derived statements and witnesses that are independent of the challenge statement and witness, our statement randomization technique does not change the witness at all. We thus have to introduce novel statistical techniques to make sure that this correlation between challenge and derived statements is not problematic overall. Finally, since we only consider a decisional PCA oracle that outputs a single bit, we cannot rely on existing techniques to deal with rewinding reductions via algebraic one-time MACs. Instead we use an entirely novel technique that considers 3κ ciphertext/plaintext pairs per query.

Restrictions. To obtain our impossibility result, our definition of SHPKE schemes requires that valid ciphertexts (and valid public keys) can efficiently be recognized using public values. Existing PKE schemes that provide strong security guarantees like IND-CCA2 security, usually do not provide such algorithms. On the contrary, their security arguments often rely on the very fact that it is hard to separate valid from invalid ciphertexts [12]. Taking this one step further, the recent technique of Libert [32] specifically introduces restrictions on the message space of existing SHPKE schemes, that always (artificially) partition the ciphertext space into valid and non-valid ciphertexts, thus sidestepping our results.

[4]For re-randomizable relations there exist efficient algorithms that can be used to uniformly draw from the class of witnesses for some statement once a single witness has been obtained. This makes sense if there is more than one witness per statement to begin with.

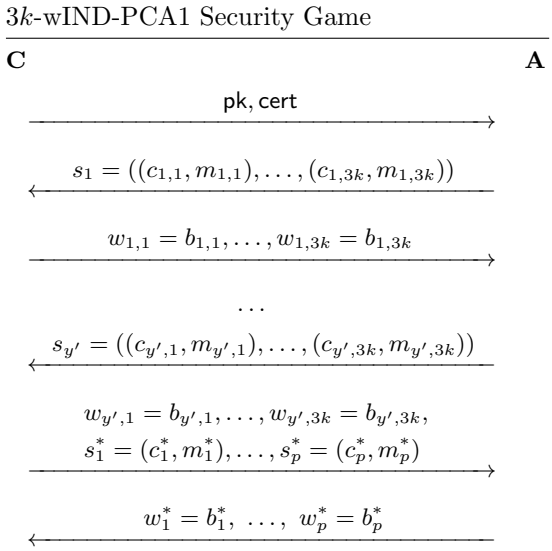

Figure 2: The $3k$-wIND-PCA1 security game between challenger **C** and attacker **A**. On initialization, a public key pk together with a certificate cert of an SHPKE are sent to the attacker. The attacker may query y' $3k$-PCA queries of the form $s_i = ((c_{i,1}, m_{i,1}), \ldots, (c_{i,3k}, m_{i,3k}))$ for $i \in [y']$. The challenger will respond with $w_{i,j} \in \{0,1\}$ for each pair $(c_{i,j}, m_{i,j})$. After y' queries, the attacker is given p challenge statements $s_1^* = (c_1^*, m_1^*), \ldots, s_p^* = (c_p^*, m_p^*)$, for which it has to determine the corresponding validity bits $w_1^*, \ldots, w_p^*$. Since it is less adaptive, this security notion is weaker than the wIND-PCA security game that admits y queries in the learning phase in case $y = 3ky'$. In our final proof we will set k to equal the security parameter $k = \kappa$.

2 Preliminaries

Definitions and Notation. We let κ denote the security parameter. Let $\mathcal{PK}, \mathcal{SK}$ be the public-key space and secret-key space respectively. Let $\mathcal{C}$ be the set of ciphertexts and let $\mathcal{M}$ be the set of plaintexts. Let $s = (c, m) \in \mathcal{C} \times \mathcal{M}$ be a ciphertext/plaintext pair, which we often refer to as a statement. The corresponding witness to a statement $s = (c, m) \in \mathcal{C} \times \mathcal{M}$ is denoted by $w \in \{0, 1\}$. We often refer to w as the validity bit of (c, m). The validity bit is 1 if c decrypts to plaintext m, otherwise it is set to 0. We let VAL denote the set of ciphertext/plaintext pairs that for a given public key are valid. Correspondingly, the set of invalid ciphertext/plaintext pairs is denoted INV. We denote vectors of statements of size k, i.e. $s = ((c_1, m_1), ..., (c_k, c_k))$ as $(c, m)_k$. Similarly, we denote the (corresponding) vector containing validity bits by $(w)_{3\kappa} = (w_1, w_2, ..., w_{3\kappa})$. We assume that for both $\mathcal{C}$ and $\mathcal{M}$, there exist efficient public memberships tests to check whether $c \in \mathcal{C}$ and $m \in \mathcal{M}$. The notation $[k; n]$ for $k, n \in \mathbb{N}$ and $k \leq n$ will be used to denote the set of natural numbers $k, k + 1, \ldots, n$. For simplicity of notation, we use $[n]$ to denote $[1; n]$.

Algorithms. Let A be a probabilistic algorithm. We use the notation $y = A(x; r)$ to say that algorithm A takes as input x and randomness r while outputting y. In this way, A can be understood as a deterministic algorithm with explicit coins r. The randomness is taken from a set $D = \{0, 1\}^l$ with some polynomial $l = l(\kappa)$. Of course, we can also make the random coins implicit. In this case, we assume the random coins $r \in D$ are drawn uniformly at random. In

the remainder of the paper, we assume that any probabilistic algorithm uses random coins in $D = \{0,1\}^l$ unless explicitly specified otherwise. When T is a set, we use $t \leftarrow T$ to denote that t is drawn uniformly random from T.

Reductions. To define reductions, we mainly follow the work of [43]. A black-box reduction is a probabilistic polynomial-time (PPT) oracle algorithm. We use black-box reductions to base the security of some cryptographic primitive X on the hardness of some other cryptographic primitive Y. A black-box reduction B breaks the security of Y using an attacker A (we denote this by B^A), whenever A breaks the security of X. The security games are played between a challenger and an attacker. For each security game, the necessary conditions will be defined to express when the attacker has successfully broken the security game. We distinguish interactive and non-interactive oracles and generally consider oracles to be deterministic. At the beginning of the security game, oracles can be given random coins as input to model probabilistic behavior. Whenever B gives new random coins to an oracle A, we consider this as B *creating an instance* of A. We require that simple reductions can only provide their oracles with randomness once – at the start of the security game. General Turing reductions, in contrast, may do this several times. This models that general reductions can rewind their oracles. All algorithms are considered to be uniform in this work. In the following, we often refer to the notion of a *spot*. A spot is a state in the execution of the security game when the attacker is expected to send a query to the reduction.

Success Probability after Rewinding. An important tool to analyze the success probability of the meta-reduction after rewinding, is the splitting lemma [45]. We use the splitting lemma to lower bound the probability that the reduction will not abort after rewinding.

Lemma 1 (Splitting Lemma)**.** *Let X and Y be finite sets. Let $G \subseteq X \times Y$ be the set containing 'good' elements. For any element $(x,y) \in X \times Y$, we define k_x to be the number $y' \in Y$ such that $(x,y') \in G$. Assume there exists some $\epsilon \in [0,1]$ such that $|G| \geq \epsilon |X \times Y|$. Let $G' = \{(x,y) \in G \mid k_x \geq \epsilon/2\ |Y|\}$, the set of so called 'super-good' elements. It holds that*

$$|G'| \geq \epsilon/2\ |X \times Y|.$$

Hoeffding's Inequality. Another tool that we require is Hoeffding's inequality. It will later be used to show the existence of a useful rewinding spot. We introduce a variant for Bernoulli random variables.

Lemma 2 (Hoeffding's Inequality)**.** *Let $X_1, ..., X_n$ be independent Bernoulli random variables. We consider their sum $S_n = \sum_{i=1}^{n} X_i$. Then the following inequality holds*

$$\Pr[|S_n - E[S_n]| > t] \leq 2e^{-2t^2/n}.$$

Here $E[S_n]$ denotes the expected value of S_n.

3 Notions for PKE

In this section, we introduce the notion of semi-homomorphic public-key encryption, as well as the associated security notions that we need to prove our impossibility result. This includes the formal security notions of wIND-PCA1 and wIND-PCA2 as well as a weaker security notion that we denote by $3k$-wIND-PCA1. This weaker security notion is used to show our second main result. For completeness, we also provide definitions for the classical IND-PCA notions in Section 3.4.

3.1 Public-Key Encryption

Let us start by providing a formal notion of a public-key encryption system. A public-key encryption (PKE) system $\mathsf{PKE} = (\mathsf{PKE.KGen}, \mathsf{PKE.Enc}, \mathsf{PKE.Dec})$ consists of three algorithms.

1. $\mathsf{PKE.KGen}(1^\kappa)$. Algorithm $\mathsf{PKE.KGen}$ is a probabilistic algorithm that takes as input the security parameter in unary and outputs an asymmetric key pair $(\mathsf{pk}, \mathsf{sk}) \in \mathcal{PK} \times \mathcal{SK}$. Algorithm $\mathsf{PKE.KGen}$ is called the key-generation algorithm.

2. $\mathsf{PKE.Enc}(\mathsf{pk}, m)$. Algorithm $\mathsf{PKE.Enc}$ is a probabilistic algorithm that takes as input the public key $\mathsf{pk} \in \mathcal{PK}$ and a message $m \in \mathcal{M}$ and outputs some ciphertext $c \in \mathcal{C}$. Algorithm $\mathsf{PKE.Enc}$ is called the encryption algorithm.

3. $\mathsf{PKE.Dec}(\mathsf{sk}, c)$. Algorithm $\mathsf{PKE.Dec}$ is a deterministic algorithm that takes as input the secret key $\mathsf{sk} \in \mathcal{SK}$ and some ciphertext c (not necessarily in $\mathcal{C}$). It either outputs a message $m \in \mathcal{M}$ if $c \in \mathcal{C}$ or a dedicated error symbol $\perp$ if $c \notin \mathcal{C}$. Algorithm $\mathsf{PKE.Dec}$ is called the decryption algorithm.

We say that a PKE is perfectly correct if for all keypairs $(\mathsf{pk}, \mathsf{sk}) \leftarrow \mathsf{PKE.KGen}(1^\kappa)$, we have for all $m \in \mathcal{M}$ that $\mathsf{Pr}[\mathsf{PKE.Dec}(\mathsf{sk}, \mathsf{PKE.Enc}(\mathsf{pk}, m)) = m] = 1$. In the remainder of the paper, we only consider perfectly correct PKEs. Next, a PKE scheme is said to be certified if i) it can be efficiently verified whether $\mathsf{pk} \in \mathcal{PK}$ and ii) we have that $[\mathsf{PKE.KGen}(1^\kappa; D)]_1 = \mathcal{PK}$. In other words, we can efficiently recognize the set of public keys that is output by the key generator. In general, we assume that a certificate cert is distributed together with the public key that proves this fact with perfect soundness. (This requirement might be relaxed to statistical soundness but for simplicity, we refrain from doing this.) It is well-known that ElGamal, Paillier [40], and Damgård-Jurik [14] are either certified or can easily be made certified [33]. In the following, we will use $w := (m \stackrel{?}{=} \mathsf{PKE.Dec}(\mathsf{sk}, c))$ to denote the process of assigning 1 to w in case $m = \mathsf{PKE.Dec}(\mathsf{sk}, c)$ and 0 otherwise.

3.2 Semi-Homomorphic PKE

Our results follow directly from the properties of a semi-homomorphic PKE scheme. We formally introduce the notion in this section. We say that a public key encryption scheme $\mathsf{PKE} = (\mathsf{PKE.KGen}, \mathsf{PKE.Enc}, \mathsf{PKE.Dec})$ is homomorphic if the following properties are fulfilled.

1. For any security parameter κ and all secret key/public key pairs $(\mathsf{pk}, \mathsf{sk}) \leftarrow \mathsf{PKE.KGen}(1^\kappa)$, it is possible to define finite, cyclic groups $(G, +)$ and $(H, \cdot)$ such that the $G = \mathcal{M}$, the plaintext space, and $H = \mathcal{C}$, the ciphertext space. Also, both G and H have an efficient membership test.

2. The set G grows exponentially in the security parameter κ.

3. There exists an efficient algorithm that draws integer t uniformly at random from $t \leftarrow [|G|]$ with only statistically small error probability δ.

4. Let $m, m' \in G$ and take any scalar $t \in [|G|]$. Then it holds that

$$\begin{aligned} \mathsf{PKE.Dec}(\mathsf{sk}, \mathsf{PKE.Enc}(\mathsf{pk}, m') \cdot \prod_{i=1}^{t} \mathsf{PKE.Enc}(\mathsf{pk}, m)) &= m' + \sum_{i=1}^{t} m \\ &= m' + tm. \end{aligned}$$

5. For every $\mathsf{pk} \in \mathcal{PK}$ and each ciphertext $c \in \mathcal{C}$, there exists a plaintext m and randomness r, so that $c = \mathsf{PKE.Enc}(\mathsf{pk}, m; r)$. In other words, we have that $\mathcal{C} = \mathsf{PKE.Enc}(\mathsf{pk}, \mathcal{M}; D)$.

6. If $m \in \mathcal{M}$ and $r \in D$ are taken uniformly random, then $\mathsf{PKE.Enc}(\mathsf{pk}, m; r)$ is distributed like a uniformly sampled ciphertext $c \in \mathcal{C}$.

As pointed out in [48], the third property is useful to model setups like Paillier PKE. These properties are fulfilled by ElGamal PKE (Appendix A), Paillier PKE, Damgård-Jurik PKE and the linear encryption system introduced by Boneh, Boyen, and Shacham [6] as well as its generalizations based on arbitrary Matrix-DDH assumptions [18]. Since we only consider certified and perfectly correct PKEs, our requirement in 1. and 5. guarantee that we can always efficiently check from the public parameters whether a purported ciphertext c indeed decrypts to some (unique) message $m \in \mathcal{M}$.

As such, our definition does not cover schemes like Damgård's ElGamal PKE [13, 34] or (the lite version of) the Cramer-Shoup PKE [12] where efficient public membership tests for recognizing valid ciphertexts do not exist. Similarly, the schemes considered in the recent work of Libert [32] sidestep our requirements. Essentially, this is because the variants in [32] require that ciphertexts are only decrypted in case the plaintext has a specific format (e.g. if it lies in some small interval). However, whether this is true cannot be recognized efficiently via public parameters only. We believe that it is possible to relax our rather strict requirements (including perfect correctness) to hold with all but statistically low error probability without violating our impossibility results, albeit under some well-defined conditions only. Specifically, we have to require that whenever an error event happens (e.g. where $\mathsf{pk} \notin \mathcal{PK}$ or $c \notin \mathcal{C}$ will remain unrecognized), the attacker can always efficiently break the security of the scheme. Jumping ahead, our impossibility result should still hold then, since for each error we then have an efficient attack that *both* ideal attacker and meta-reduction can launch. As a consequence, error events can efficiently be simulated by the meta-reduction as well.

3.3 Algorithms for Randomization, Sampling, and Random Sampling

Let us now also define two algorithms for generating and randomizing (in)valid ciphertext/plaintext pairs in SHPKE schemes. Their properties readily follow from the properties of the SHPKE. These algorithms will later be used by the attacker and meta-reduction as described in Section 6.

Randomization. The probabilistic randomization algorithm $\mathsf{Randomize}$ takes as input the public key pk, a ciphertext c, and a plaintext m. Let w be the validity bit of (c, m). The output of $\mathsf{Randomize}(\mathsf{pk}, c, m)$ is a ciphertext/plaintext pair (c', m') with validity bit w', such that $w = w'$. The randomization algorithm guarantees that (c', m') is indistinguishable from any uniformly random pair (c'', m'') with equal validity bit (so $w' = w''$). Let us describe the randomization algorithm concretely. Let $\mathsf{pk} \in \mathcal{PK}$. For any pair (c, m), we perform the following.

- We generate uniformly random $r \leftarrow G$ and uniformly random $s \leftarrow [|G|]$ with overwhelming probability $1 - \delta$ (according to Property 3).
- We set $c' = c^s \cdot \mathsf{PKE.Enc}(\mathsf{pk}, r)$ and $m' = sm + r$.
- Finally, we output (c', m').

The validity of the ciphertext/plaintext pairs is preserved by the properties of the SHPKE scheme: if $w = 1$, i.e. (c, m) is a valid ciphertext/plaintext pair, we have $c' = \mathsf{PKE.Enc}(\mathsf{pk}, m)^s \cdot \mathsf{PKE.Enc}(\mathsf{pk}, r)$ and $m' = sm + r$. Thus we have $w' = 1$ as well. Similarly, if $w = 0$, we have that

$c' = \mathsf{PKE.Enc}(\mathsf{pk}, m)^s \cdot \mathsf{PKE.Enc}(\mathsf{pk}, r)$ and $m' = s(m+z)+r = (sm+r)+sz$ for some $z \neq 0$. Since $s \in [|G|]$ we have in particular $s \neq 0$. Thus $zs \neq 0$ and $w' = 0$ as well. Moreover, as both r and s are chosen independently and uniformly at random (with overwhelming probability), we have that $c', m' \in \mathcal{C} \times \mathcal{M}$ is a uniform random element in VAL if $w = 1$ with probability at least $(1-\delta)$, because c' is ranging over every possible ciphertext. Similarly, if $w = 0$, then (c', m') is a uniform random element in INV. In this case, we have that $c' = \mathsf{PKE.Enc}(\mathsf{pk}, m)^s \cdot \mathsf{PKE.Enc}(\mathsf{pk}, r)$ and $m' = s(m+z)+r = (sm+r)+sz$ for some $z \neq 0$. Since both $r \in G$ and $s \in [|G|]$ are drawn at random and since in particular $s \neq 0$, we have that c' ranges over all possible ciphertexts and m' ranges over all possible plaintext conditioned on the fact that (c', m') is an invalid ciphertext/plaintext pair.

Sampling and Random Sampling. Let us now introduce two probabilistic algorithms Sample and $\mathsf{RSample}$ that we will often make use of in the following.

The algorithm Sample takes as input public key pk and bit w. To evaluate $(c, m) \leftarrow \mathsf{Sample}(\mathsf{pk}, w)$ it executes the following steps.

1. First, it draws two messages $m', m'' \in \mathcal{M}$ uniformly at random.
2. After that, it draws fresh random coins r and computes the ciphertext $c = \mathsf{PKE.Enc}(\mathsf{pk}, m'; r)$. If $w = 1$, it sets $m = m'$. Otherwise, it sets $m = m''$.
3. Finally, it draws fresh random coins r' and outputs $\mathsf{Randomize}(\mathsf{pk}, c, m; r')$[5] and w.

Using Sample, we can produce ciphertext/plaintext pairs (c, m) with a given validity bit w such that (c, m) are drawn uniformly random in the set VAL or INV depending on w.

The probabilistic random sampling algorithm $\mathsf{RSample}$, takes as input the public key pk. It creates a ciphertext/plaintext pair together with its validity bit $(c, m, w) \leftarrow \mathsf{RSample}(\mathsf{pk})$, such that the probability of the event $w = 1$ is $1/2$. The algorithm generates these values by drawing $w \leftarrow \{0, 1\}$ and simply outputting $(c, m) \leftarrow \mathsf{Sample}(\mathsf{pk}, w)$ together with w. This shows that independent of the sizes of VAL and INV we can draw statements uniformly from either set with probability $1/2$. Observe that Sample and $\mathsf{RSample}$ use $\mathsf{Randomize}$, which fails with statistically low probability δ. They thus also have overwhelming success probability $(1-\delta)$.

3.4 IND-PCA, wIND-PCA, and $3k$-wIND-PCA1 Security

We introduce the classical IND-PCA security notion (also depicted in Figure 4): Indistinguishability under plaintext checkable attacks [1]. Next, we present two progressively weaker versions of this security notion called weak IND-PCA security (wIND-PCA) and $3k$-wIND-PCA1 security. The corresponding security games for the weaker versions are given in Figure 1 (wIND-PCA1), Figure 2 ($3k$-wIND-PCA1), and Figure 3 (wIND-PCA2). In the appendix, we also provide instantiations of these security notions for ElGamal PKE A in Figure 5 and Figure 6. Let us begin with the notion for IND-PCA security.

1. The challenger C computes $(\mathsf{pk}, \mathsf{sk}) \leftarrow \mathsf{PKE.KGen}(1^\kappa)$ and sends pk to the attacker A.
2. The attacker A may (adaptively) query up to y_1' queries to C. Each query consists of a ciphertext/plaintext pair $s_i = (c_i, m_i) \in \mathcal{C} \times \mathcal{M}$ for $i \in [y_1']$.
3. The challenger C responds to each query with the truth value $w_i := (m_i \stackrel{?}{=} \mathsf{PKE.Dec}(\mathsf{sk}, c_i))$.

[5] Since m', m'' are drawn randomly, applying $\mathsf{Randomize}$ is not strictly necessary. However, it simplifies the analysis later as now all queries sent to the reduction, before and after the rewinding, have been output by $\mathsf{Randomize}$.

4. The attacker computes p plaintexts $\hat{m}^*_{1,1}, \ldots, \hat{m}^*_{1,p}$ and sends them to the challenger.

5. Challenger C determines p challenge statements as follows. For each $j \in [p]$, C draws random message $\hat{m}^*_{0,j}$ and a random bit w^*_j and computes $c^*_j \leftarrow \mathsf{PKE.Enc}(\mathsf{pk}, \hat{m}^*_{d,j})$ with $d = 1 - w^*_j$. Next, C sets $m^*_j = \hat{m}^*_{0,j}$ and outputs $s^*_j = (c^*_j, m^*_j)$ to the attacker A.

6. The attacker A may (adaptively) query up to y'_2 queries to C. Each query consists of a ciphertext/plaintext pair $s_i = (c_i, m_i) \in \mathcal{C} \times \mathcal{M}$ for $i \in [y'_2]$.

7. A responds with bits $w'^*_1, \ldots, w'^*_p$.

We say the attacker wins if $\bigwedge_{i=1}^{p}(w'^*_i = w^*_i)$. Moreover, we say that the PKE scheme is IND-PCA secure if the *advantage*

$$\left| \mathsf{Pr}\left[\bigwedge_{i=1}^{p} (w'^*_i = w^*_i) \right] - 1/2^p \right|$$

is negligible for any PPT A. We also say the attacker breaks the security of a PKE scheme if its advantage is non-negligible. Additionally, we say that the PKE scheme is wIND-PCA secure if we collapse Steps 4. and 5. of the IND-PCA game and substitute it by the following step, as depicted in Figure 1.

4*. Challenger C determines p challenge statements as follows. For each $j \in [p]$, C draws two random messages $\hat{m}^*_{0,j}$ and $\hat{m}^*_{1,j}$ and a random bit w^*_j and computes $c^*_j \leftarrow \mathsf{PKE.Enc}(\mathsf{pk}, \hat{m}^*_{1,j})$. Challenger C sets ${m_j}^* = \hat{m}^*_{w^*,j}$ and outputs $s^*_j = (c^*_j, m^*_j,)$ to the attacker A.

As is common, we introduce subnotions, making distinctions between IND-PCA1 and IND-PCA2 security on the one hand and wIND-PCA1 and wIND-PCA2 on the other hand. In traditional notions like IND-CCA security, p is usually set to $p = 1$. Having $p > 1$ generalizes this concept, making it harder for the attacker since it has to deliver several correct solutions at the same time.[6] The advantage takes into account that, for any p, the attacker might just guess the validity bits. Practically, this has been used in modelling the threat of mass surveillance [5]. Using established terminology, this results in the natural AND notion [5] for suitable parameter choices. In case the attacker cannot reveal the challenge bits of any of the challenges, this is polynomially equivalent to other notions like LORX, RORX, UKU. Interestingly, as our proofs show, the techniques we use can vary in the range of parameters for p that they support. In particular, in Appendix B we also provide an IND-PCA1 impossibility result for simple reductions that, however, requires p to be a constant, whereas our more general result in Section 6 supports arbitrary polynomial p. Depending on whether we have at least one IND-PCA2 query per challenge or not, we consider the following cases.

- In the IND-PCA2 game, we have that $y'_2 \geq p$.
- In the IND-PCA1 game, we have that $y'_2 < p$ but $y'_1 \geq 1$.

For convenience, we call s^* challenge (statement), c^* challenge ciphertext, m^* challenge plaintext, and w^* challenge bit. We call w'^* the (attacker's) guess and $\hat{m}^*_0$ and $\hat{m}^*_1$ candidate plaintexts. If $b := (m \stackrel{?}{=} \mathsf{PKE.Dec}(\mathsf{sk}, c))$ we also say that b is the *validity bit* of $s = (c, m)$. Moreover, we often refer to the interaction in Steps 1, 2, and 3 as the learning phase of the security game.

It is easy to see that the wIND-PCA security game is weaker than IND-PCA security: in contrast to classical indistinguishability attacks, A cannot even provide a candidate plaintext.

[6] This is different from classical, hybrid arguments where the reduction expects the attacker to deliver a solution for one instance (out of many).

Instead, it is entirely chosen by the challenger. Since wIND-PCA security is weaker than IND-PCA security it is also weaker than IND-CCA security. This is because the attacker obtains less information in the PCA learning phase than in the CCA learning phase. Instead of decryption queries, the attacker can now only make PCA queries that solely output a single bit. Put differently, a decryption oracle can easily be used to implement a PCA oracle. We show our impossibility result for wIND-PCA.

We remark that the *weakest* variant of the wIND-PCA1 security game with $y' = y'_1 + y'_2$ queries overall is in case $y'_2 = 0$. This is because in this game the attacker cannot rely on the knowledge of the challenge when computing any of its queries. For more generality, we will thus in the following only consider this variant. In particular, when introducing our next security notions, we will not consider scenarios where the attacker makes queries after the challenge and always implicitly assume that $y'_2 = 0$.

This shows that our final new security notion, $3k$-wIND-PCA1 security, as depicted in Figure 2, is provably weaker than wIND-PCA security in case the attacker is allowed to send the *same overall number of ciphertext/plaintext pairs* to the challenger. In contrast to the wIND-PCA game, $3k$ of these pairs have to be sent at a time. This makes the $3k$-wIND-PCA1 game simply less adaptive. We refer to these queries as $3k$-PCA queries. In our result, we will set $k = \kappa$.

It is clear that this game is weaker than wIND-PCA1 security if $y \geq 3ky'$: any attacker A that is able to efficiently break the $3k$-wIND-PCA1 security of some PKE scheme with y' $3k$-PCA queries, is immediately able to efficiently break its wIND-PCA1 security with y PCA queries as well in case $y \geq 3ky'$.

4 Interactive Complexity Assumptions

t-Interactive Complexity Assumptions. Our impossibility result excludes reductions that reduce wIND-PCA security to that of arbitrary *interactive* complexity assumption. We adopt the general notion of a t-interactive complexity assumption (tICA) from [48], which is based on the formulation in [3]. We almost take the definition in verbatim. A tICA consists of $t + 3$ Turing machines, given by $\mathsf{ICA} = (\mathsf{ICAGen}, \mathsf{ICAVer}, \mathsf{ICATriv}, \mathsf{ICAQuery}_1, \ldots, \mathsf{ICAQuery}_t)$. We provide a description for each of these Turing machines.

- ICAGen. Algorithm ICAGen is an efficient probabilistic instance generator. As input it takes the security parameter κ and it outputs a problem instance c and an initial state st_0.
- $\mathsf{ICAQuery}_1$ – $\mathsf{ICAQuery}_t$. Algorithm $\mathsf{ICAQuery}_i$, given as input the i-th query q_i and state st_{i-1} for $i \in [t]$, efficiently outputs a response p_i and state st_i. We consider the set of all $\mathsf{ICAQuery}_i$ algorithms as a stateful algorithm $\mathsf{ICAQuery}$.
- $\mathsf{ICATriv}$. Algorithm $\mathsf{ICATriv}$, given a problem instance c and t-time oracle access to $\mathsf{ICAQuery}$, uses a trivial attack strategy to efficiently determine a candidate solution s to c.
- ICAVer. Algorithm ICAVer, given a problem instance c, the final state st_t, and a solution s, outputs a bit b. If $\mathsf{ICAVer}(c, st_t, s) = 1$, then s is a valid solution to problem instance c. If the ICAVer algorithm is efficient, then ICA is said to be falsifiable.

Security Games for tICAs. Let A be the attacker and N be a t-interactive complexity assumption. We present a general description for a security game ICA_N^A.

1. The game starts by running $(c, st_0) \leftarrow \mathsf{ICAGen}(1^\kappa)$.

2. Attacker A is given c, together with t-time oracle access to $\mathsf{ICAQuery}$.

3. The attacker A outputs a candidate solution s.

4. The experiment returns $\mathsf{ICAVer}(c, st_t, s)$.

We provide a definition for breaking the security of tICA. Intuitively, attacker A wins the security game if it returns a valid candidate solution s with a probability non-negligibly better than any trivial attack. Moreover, A has to provide s in polynomial time. An example of such a trivial attack is simply guessing $b \in \{0, 1\}$, which gives the trivial advantage of $1/2$ in an indistinguishability-based security game.

Definition 1. Let A be an attacker. Attacker A is said to efficiently break tICA ICA if A has a polynomial running time and if the advantage

$$|\mathsf{Pr}[\mathsf{ICA}^{A}_{\mathsf{ICA}}(1^{\kappa}) \Rightarrow 1] - \mathsf{Pr}[\mathsf{ICA}^{\mathsf{ICATriv}}_{\mathsf{ICA}}(1^{\kappa}) \Rightarrow 1]| = \epsilon$$

is non-negligible. The probability is taken over the random coins used by the probabilistic algorithms $\mathsf{ICAGen}, \mathsf{ICATriv}, \mathsf{ICAQuery}$, as well as A. A tICA ICA is said to be secure if no probabilistic polynomial algorithm can break ICA.

5 Impossibility of wIND-PCA2 Security

As warm-up, let us first investigate wIND-PCA2 security for SHPKE schemes.

Theorem 1. *There is no SHPKE scheme that is wIND-PCA2 secure.*

The proof of this theorem follows ElGamal's well-known IND-CCA2 impossibility result. In contrast to the existing impossibility result, we not only exploit the malleability of the ciphertexts. We rather rely on the fact that in SHPKE schemes ciphertexts and plaintext pairs can be randomized without changing the validity bit. To this end we can use the $\mathsf{Randomize}$ procedure that works for *pairs* of ciphertexts and plaintexts. Moreover, we have to consider that the attacker has to provide answers to p queries overall.

Proof. Let $\mathsf{PKE} = (\mathsf{PKE.KGen}, \mathsf{PKE.Enc}, \mathsf{PKE.Dec})$ be an SHPKE scheme. Consider the following wIND-PCA2 attacker A. Assume A receives the public key pk from the challenger C. The attacker A makes y'_1 arbitrary PCA queries and receives the corresponding responses. (For example, the attacker could send the pair (c, m) a total of y'_1 times to the challenger where c is the encryption of random message m.) When A receives the challenges $s^*_1 = (c^*_1, m^*_1), \ldots, s^*_p = (c^*_p, m^*_p)$, the attacker A computes $(c'_i, m'_i) \leftarrow \mathsf{Randomize}(\mathsf{pk}, c^*_i, m^*_i)$ to generate p derived statements $s'_i = (c'_i, m'_i)$ for $i \in [p]$. Next, A sends $s'_1, \ldots, s'_p$ to C one after the other. Since in the wIND-PCA2 game we have $y'_2 \geq p$, the attacker can always do this by using up the y'_2 queries available after the challenges have been received. To each $s'_i = (c'_i, m'_i)$, the challenger responds with the corresponding validity bit b'_i. Because $\mathsf{Randomize}$ produces outputs that have the same validity bit as the input, b'_i is also a correct validity bit for s^*_i. If $y'_2 > p$, A makes $y'_2 - p$ additional arbitrary queries to C and receives back the responses. Finally, A sends $b'_1, \ldots, b'_p$ to C. In this way, the attacker will always win in the security game. Observe that all arguments are independent of the concrete value of y'_1. □

In this proof, we rely on the $\mathsf{Randomize}$ algorithm that we have defined before. This is for simplicity. We remark that the strong hiding property of $\mathsf{Randomize}$ (outputs are always uniformly random in the set VAL respectively INV and thus only provide very limited information

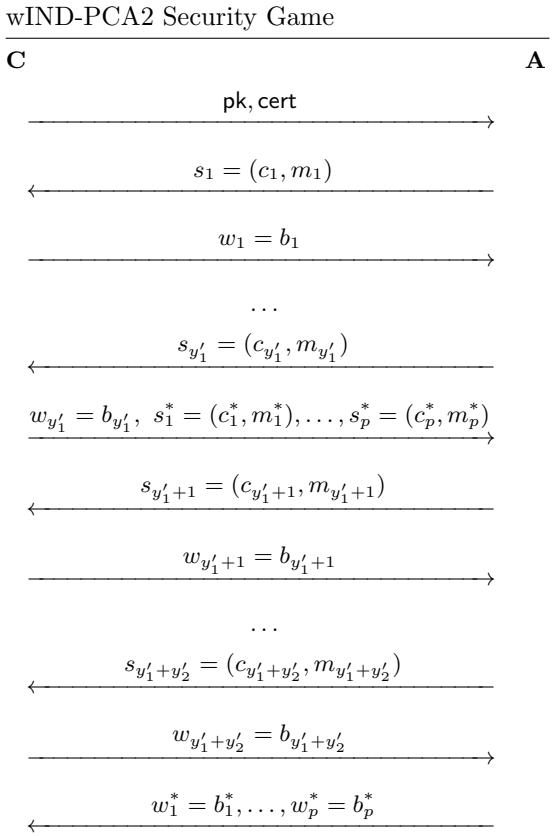

Figure 3: The wIND-PCA2 security game between challenger **C** and attacker **A**.

on the input statement) is not necessary. Indeed, it seems the proof can rely on Property 4. and Property 5. as given in the general definition of SHPKE in Section 3.2. Also, observe that the proof essentially relies on the fact that all derived statements can be sent to the challenger *after* the challenge has been received. This strategy does not work in the case of wIND-PCA1 security. We present the following corollary, which is a direct result of Theorem 1 since IND-PCA2 security implies wIND-PCA2 security.

Corollary 1. *There is no SHPKE scheme that is IND-PCA2 secure.*

6 Impossibility of wIND-PCA1 Security

Let us now consider the more challenging case of wIND-PCA1 security. In our second main result, we rule out that SHPKE schemes can be shown 3κ-wIND-PCA1 secure via general Turing reductions. We use this result to finally rule out provable wIND-PCA1 security for SHPKE schemes. Since wIND-PCA1 is weaker than IND-PCA security, the impossibility result immediately transfers to IND-PCA security as well.

Theorem 2. *Let κ be the security parameter. Let* PKE *be a certified SHPKE. Let p, the number of challenges statements that the attacker must solve in the 3κ-wIND-PCA1 security game, be polynomial. Then, there exists no PPT reduction B that reduces the 3κ-wIND-PCA1 security of*

PKE *with* $2t + 2\kappa + 2u$ *adaptive PCA queries to the security of any* t*ICA, while creating no more than* u *attacker instances for* $t + u < O(\kappa^2)$.

In Appendix B, we present a more restricted but conceptually much simpler result that rules out simple reductions. This result has better quantitative bounds. On the other hand, however, it only applies to scenarios with a constant number of challenges p. Intuitively, the underlying reason is that when dealing with simple reductions, our meta-reduction simply hopes that the reduction only provides correct responses in the learning phase after it rewinds the reduction. However, the meta-reduction cannot check if this is actually true. Nevertheless, with non-negligible probability this is bound to happen for a single challenge since the reduction has, by assumption, non-negligible success probability. In this way, the first challenge w_1^* can easily be computed by the meta-reduction with non-negligible probability. However, the meta-reduction uses this process for all other challenges as well and the overall probability that the reduction always provides correct answers after all the rewindings diminishes exponentially in p. This problem does not occur in the proof of Theorem 2 since after each rewinding we can actually check with *overwhelming probability* whether the reduction has output a correct response to our rewound query. A simple application of Bernoulli's inequality shows that the overall probability remains overwhelming after polynomially many challenges.

Remaining Challenges. To prove Theorem 2, we have to find solutions for the following challenges when dealing with general Turing reductions B.

1. B may rewind the adversary A.
2. B may communicate externally with the tICA challenge (at most t times).
3. B may initiate u attacker instances of A.
4. B may provide incorrect responses.

In the introduction, we have already sketched how we deal with the latter three challenges. It remains to describe how we deal with rewinding reductions more generally. Essentially, the main idea is to make the ideal attacker behave deterministically towards the reduction. Since the meta-reduction controls the reduction, it can always simulate such a behavior by proper bookkeeping. If the ideal attacker A behaves deterministically (in the initially provided random coins), any reduction that rewinds A to some earlier point will receive the same queries as before in the learning phase. This limits the benefits that the reduction obtains by being able to rewind the attacker. The only way a reduction can change the attacker's behavior in the learning phase at all is by either creating a new instance of A (via new initial randomness) or by responding incorrectly to some of the queries. However, slightly jumping ahead, we will have that, by setup, the ideal attacker knows the correct responses to begin with. To make the attacker deterministic, we follow previous works like [43, 39, 38] that give A access to some (private) random oracle $h(\cdot)$. We assume that each instance of A is given some initial random coins r (by B) at the beginning. Every instance of A will now apply $h(\cdot)$ to their transcript so far of sent and received messages to generate fresh random coins for all the randomized algorithms it uses next. We will not make this explicit. However, we emphasize that for each query sent by A in the learning phase, there is only a single correct validity bit that B can send without the attacker aborting. Therefore, the entire learning phase is now essentially fixed for each attacker instance when it receives the initial randomness.

Stateless Attackers: The Next-Message Function. As is common in meta-reduction-based results, we will model the attacker as a stateless next-message function that is given as input the entire communication transcript so far (including the initial randomness). Intuitively, this outsources the state management to the reduction. Moreover, it gives her full control of the state that the adversary should be in next via a single interface. Creating new instances corresponds to using new initial randomness. However, to make sure that the attacker only accepts plausible transcripts, we require it to check all the steps in the transcript by recomputing its values. If the transcript does not correspond to a plausible transcript the attacker aborts. Otherwise, it outputs the next message, which can either be the next query in the learning phase or the response to the challenge. This modeling simplifies arguments about the view and behavior of the reduction: essentially view and behavior can now not depend on a state that A is in, simply because A is stateless. To be consistent with this modeling, we assume in the following that B sends entire transcripts to A. For simplicity, we will often not consider this mechanism explicitly and instead view the attacker as stateful.

Useful Rewinding Spot. Our proof strategy is centered around the idea of guaranteeing that a useful rewinding spot exists. By this, we refer to a state in the learning phase of B that the meta-reduction rewinds B back to. More precisely, this is a state of B immediately before A outputs a new query to the reduction. A useful rewinding spot has the following properties: i) no external queries are made to the tICA by B before it responds to A's query, ii) the response of B is correct, and iii) B does not wait for any of the other $u-1$ attacker instances to respond to a challenge statement before B outputs the response to A's query.

Sampling Vectors of Ciphertext/Plaintext Pairs. To prepare our description, let us now describe a probabilistic algorithm $\mathsf{VectorSample}$ that, given input pair $(c, m) \in \mathcal{C} \times \mathcal{M}$, outputs $3k$ ciphertext/plaintext pairs $(c_1, m_1), \ldots, (c_{3k}, m_{3k})$ together with the description of a random permutation π on the set $\{1, \ldots, 3k\}$. In the following, assume that w is the validity bit of (c, m). We stress however, that w is not given as input to $\mathsf{VectorSample}$. The algorithm works as follows:

1. First, $\mathsf{VectorSample}$ generates two additional ciphertext/plaintext pairs that are computed via $(c'_1, m'_1) \leftarrow \mathsf{Sample}(\mathsf{pk}, 1)$ and $(c'_2, m'_2) \leftarrow \mathsf{Sample}(\mathsf{pk}, 0)$. By construction, the first pair has validity bit 1 while the second has validity bit 0. Additionally, $\mathsf{VectorSample}$ sets $(c'_3, m'_3) := (c, m)$.

2. Next, $\mathsf{VectorSample}$ creates $3k$ pairs $(c''_1, m''_1), \ldots, (c''_{3k}, m''_{3k}) \in \mathcal{C} \times \mathcal{M}$ by calling $(c''_i, m''_i) \leftarrow \mathsf{Randomize}(\mathsf{pk}, c'_{\lceil i/k \rceil}, m'_{\lceil i/k \rceil})$ for all $1 \leq i \leq 3k$. Intuitively, this creates $3k$ random pairs such that the first k (for $1 \leq i \leq k$) of them have validity bit 1, the central k pairs (for $k+1 \leq i \leq 2k$) have validity bit 0, and the last k pairs (for $2k+1 \leq i \leq 3k$) have the same validity bit as the input pair, i.e. validity bit w.

3. In the last step, $\mathsf{VectorSample}$ draws a random permutation π on $[3k]$.

4. Finally, $\mathsf{VectorSample}$ outputs $(c_1, m_1), \ldots, (c_{3k}, m_{3k})$ together with π where the positions of the (c_j, m_j) are permuted according to π as follows: $(c_j, m_j) := (c''_{\pi(j)}, m''_{\pi(j)})$.

When $((c_1, m_1), \ldots, (c_{3k}, m_{3k}), \pi) \leftarrow \mathsf{VectorSample}(\mathsf{pk}, c, m)$ is called, we always have by the properties of $\mathsf{Randomize}$ that exactly $2/3$ of the (c_i, m_i) have validity bit w. Moreover, using π we can always determine which of the pairs (c_i, m_i) have been derived from (c'_1, m'_1) and (c'_2, m'_2) and accordingly, what their validity bit is. Also, we can compute all the (c_i, m_i) that have been derived from the input pair. Importantly, all these pairs must always have the same validity bit.

Testing Vectors of Ciphertext/Plaintext Pairs. This gives rise to an efficient deterministic validity test VectorTest that on input $(c_1, m_1), \ldots, (c_{3k}, m_{3k}) \in \mathcal{C} \times \mathcal{M}$, permutation π on $[3k]$, and the purported vector of validity bits $w_1, \ldots, w_{3k} \in \{0, 1\}$, tests whether all the w_i indeed correspond to the validity bits of (c_i, m_i). We design a subroutine VectorTest to compute bit $b \leftarrow \mathsf{VectorTest}(\mathsf{pk}, (c_1, m_1), \ldots, (c_{3k}, m_{3k}), w_1, \ldots, w_{3k}, \pi)$. This algorithm does the following:

1. It computes the three index sets $I_j = \{\pi(\ell) | 1 + (j-1)k \leq \ell \leq jk\}$ for $j = 1, 2, 3$.
2. Next, VectorTest checks whether it holds for all positions $i \in I_1$ that $w_i = 1$.
3. After that, it checks whether for all positions $i \in I_2$ it holds that $w_i = 0$.
4. Finally, VectorTest checks whether for all positions $i \in I_3$ it holds that the w_i are all equal, i.e. if $|\{w_i | i \in I_3\}| = 1$.
5. If all the checks in Steps 2, 3, and 4 are successful, VectorTest outputs 1. Otherwise, it outputs 0.

We observe that in case that $((c_1, m_1), \ldots, (c_{3k}, m_{3k}), \pi)$ is the output of VectorSample, there are at most two possible bit vectors $w_1, \ldots, w_{3k}$ for which we have that the invocation of $\mathsf{VectorTest}((c_1, m_1), \ldots, (c_{3k}, m_{3k}), \pi, w_1, \ldots, w_{3k})$ makes the output 1. These two vectors differ exactly in the k positions $i \in I_3$. One of these vectors contains all the correct validity bits. The other one has all the validity bits w_i with $i \in I_3$ flipped. (Otherwise the check in Step 4 of VectorTest would fail.) Moreover, given only $(c_1, m_1), \ldots, (c_{3k}, m_{3k})$, it is hard to compute I_3. To see this, assume that 2/3 of the (c_i, m_i) have validity bit 1. (The argument for $w = 0$ works analogously.) Information-theoretically, this decreases the set of possible indices that I_3 may contain to $2k$ candidates. This set of candidates not only consists of the pairs that have been derived from (c'_3, m'_3) but also of those that have been derived from (c'_1, m'_1) when executing VectorSample. However, since the output of Randomize is always uniformly random in VAL the pairs at positions $i \in I_1$ are distributed exactly like those that are located at positions $i \in I_3$. Thus, the best the reduction can do is to guess the k positions of I_3. However, this has statistically low success probability $\binom{2k}{k}^{-1}$.

6.1 The Ideal Attacker

In this section we present the ideal attacker A for the 3κ-wIND-PCA1 security game. Let C be the challenger of the security game. Let u be the maximum number of attacker instances the reduction is allowed to initiate. For simplicity of notation, let $y = 2t + 2\kappa + 2u$.

1. *Initialization.* On initialization, the challenger sends a public key pk and a proof cert, together with random coins r, to A. The ideal attacker A checks whether pk, cert are valid. If not, A aborts. Otherwise, it computes $(c, m, w) \leftarrow \mathsf{RSample}(\mathsf{pk})$. Then, it calls the algorithm VectorSample to compute $((c_1, m_1)_{3\kappa}, \pi_1) \leftarrow \mathsf{VectorSample}(\mathsf{pk}, c, m)$. Finally, it sends $(c_1, m_1)_{3\kappa}$ to C.
2. *Verifying Responses of B.* In case A receives an input that is structured according to the following form $T = (\mathsf{pk}, \mathsf{cert}, r, (c_1, m_1)_{3\kappa}, (w'_1)_{3\kappa}, \ldots, (c_i, m_i)_{3\kappa}, (w'_i)_{3\kappa})$ for $i \in [y-1]$, A checks whether T is plausible. To this end, it computes all the $(c_i, m_i)_{3\kappa}$ and checks whether they are equal to the values in T. To recompute $(c_i, m_i)_{3\kappa}$, A proceeds as before. It calls $(c, m, w_i) \leftarrow \mathsf{RSample}(\mathsf{pk})$ and after that $((c_i, m_i)_{3\kappa}, \pi_1) \leftarrow \mathsf{VectorSample}(\mathsf{pk}, c, m)$. (Recall that implicitly the randomness used in the probabilistic algorithms is actually derived deterministically.) If any of the recomputed values is different from the one in T,

A aborts. At the same time, A verifies if the received validity bits are indeed correct. To this end, it checks for each i whether, $\mathsf{VectorTest}(\mathsf{pk}, (c_i, m_i)_{3\kappa}, \pi, (w'_i)_{3\kappa}) = 1$. Moreover, A checks if the j-th component of $(w'_i)_{3\kappa}$ for $j = \pi(2\kappa + 1)$ is equal to w_i. On failure, attacker A aborts. Finally A computes a new query $(c_{i+1}, m_{i+1})_{3\kappa}$ as before and sends it to the challenger.

3. *Computing the challenge validity bits.* If A receives an input that is structured according to the following form $T = (\mathsf{pk}, \mathsf{cert}, r, (c_1, m_1)_{3\kappa}, (w'_1)_{3\kappa}, \ldots, (c_y, m_y)_{3\kappa}, (w'_y)_{3\kappa}, s^*_1, \ldots, s^*_p)$, the attacker A first checks, in the same way as before, whether the transcript prefix $T' = (\mathsf{pk}, \mathsf{cert}, r, (c_1, m_1)_{3\kappa}, (w'_1)_{3\kappa}, \ldots, (c_y, m_y)_{3\kappa}, (w'_y)_{3\kappa})$ is plausible. Next, A uses its unbounded computational resources to compute the validity bits $w^*_1, \ldots, w^*_p$ for the challenges $s^*_1, \ldots, s^*_p$. To this end, it can, for example, find for each s^*_i the randomness r_i such that $(s^*_i, w^*_i) \leftarrow \mathsf{RSample}(\mathsf{pk}; r_i)$. Finally, A outputs $w^*_1, \ldots, w^*_p$.

We remark that A, due to having unbounded computational resources, wins the security game with overwhelming probability.

6.2 The Meta-Reduction

In this section, we present the meta-reduction that simulates the ideal attacker. Notation-wise, the z-th instance of A is denoted by $A[z]$. Any variable x belonging to the communication between B and $A[z]$ is denoted by $x[z]$. We require that $A[z]$ can store the internal state of B to allow M to rewind B.

As sketched before, we exploit that M runs B and thus can easily simulate the deterministic behavior of $A[z]$ by proper bookkeeping. As an example, M may use precomputations to generate all random values (that are computed by the ideal attacker via $h(\cdot)$) beforehand and keeps track of their usage in a table. In this way, they can be used consistently. In the following, we will assume that the meta-reduction will do this implicitly throughout the entire simulation.

The meta-reduction M will, in each simulated attacker instance, receive the initial values from the reduction and start to generate $3k$-PCA queries. To each query, it receives the corresponding response from the reduction. This will continue until for one of the u instances, say instance $z \in [u]$, $A[z]$ has to provide a response to its challenges. As a response, M will repeatedly rewind B, to obtain valid responses stepwise for all the $s^*_i[z]$ with $i \in [p]$. For each i the meta-reduction sends a ciphertext/plaintext pair to B that is derived from $s^*_i[z]$. This derived statement is sent at a random position in the learning phase of instance z. The meta-reduction hopes that this position turns out to be a useful rewinding spot. In this case instance z will receive back a response that is correct while no other instances have been asked by to provide challenges and no query has been asked to the tICA. All these three conditions can efficiently be checked by M since it controls the execution of B. Moreover, to make sure the response is correct, it can apply $\mathsf{VectorTest}$. If the output is 1, then M has that with overwhelming probability, the response is indeed correct. If M does not find a useful rewinding spot, it repeats the process, starting with a fresh randomization of s^*_i.

Let us present the construction of meta-reduction M more formally. For simplicity of notation, let $y = 2t + 2\kappa + 2u$.

0. The meta-reduction M receives the tICA instance c and relays it to B along with random coins $r_B \in D_B$.

1. *Initialization.* On initialization of some attacker instance z, B sends a public key $\mathsf{pk}[z]$ and a proof $\mathsf{cert}[z]$, together with random coins $r[z]$, to $A[z]$. Then, $A[z]$ checks whether $\mathsf{pk}, \mathsf{cert}$ are valid. If not, M aborts the simulation of $A[z]$. Otherwise, $A[z]$ computes

$(c_1[z], m_1[z], w_1[z]) \leftarrow \mathsf{RSample}(\mathsf{pk})$. Then, to construct a 3κ query, it calls the subroutine $((c_1, m_1)_{3\kappa}[z], \pi_1[z]) \leftarrow \mathsf{VectorSample}(\mathsf{pk}[z], c_1[z], m_1[z])$. Note that M keeps track of the number of attacker instances that have been initiated by B. For rewinding purposes, M stores the execution state of B as $\mathsf{state}_1[z]$. Next it sends $(c_1, m_1)_{3\kappa}[z]$ to B.

2. *Verifying responses of B.* For attacker instance z, if $A[z]$ receives input value $T[z] = (\mathsf{pk}[z], \mathsf{cert}[z], r[z], (c_1, m_1)_{3\kappa}[z], (w'_1)_{3\kappa}[z], \ldots, (c_j, m_j)_{3\kappa}[z], (w'_j)_{3\kappa})[z]$ for $j \in [y-1]$, $A[z]$ checks whether $T[z]$ is plausible. To this end, it computes all the $(c_i, m_i)_{3\kappa}[z]$ with $i \in [j]$ on its own and checks whether they are equal to the values in $T[z]$. To recompute $(c_i, m_i)_{3\kappa}[z]$, $A[z]$ proceeds as the ideal attacker. It calls $(c[z], m[z], w_i[z]) \leftarrow \mathsf{RSample}(\mathsf{pk}[z])$ and uses the result to compute $((c_i, m_i)_{3\kappa}[z], \pi_i[z]) \leftarrow \mathsf{VectorSample}(\mathsf{pk}[z], c[z], m[z])$. If one of the recomputed values is different from the one in $T[z]$, M aborts the simulation of $A[z]$. At the same time, $A[z]$ verifies if the received validity bits are indeed correct. To this end, it checks for each i whether $\mathsf{VectorTest}(\mathsf{pk}[z], (c_i, m_i)_{3\kappa}[z], \pi_i[z], (w'_i)_{3\kappa}[z]) = 1$. Moreover, $A[z]$ checks if the $\pi(2\kappa+1)[z]$-th component of $(w'_i)_{3\kappa}[z]$ is equal to $w_i[z]$. If this check fails, $A[z]$ aborts. Next, $A[z]$ generates a new query $(c_{i+1}, m_{i+1})_{3\kappa}[z]$ in the same way as before. For rewinding purposes, $A[z]$ stores the execution state of B as $\mathsf{state}_{i+1}[z]$. Finally, it sends $(c_{i+1}, m_{i+1})_{3\kappa}[z]$ to B.

3. *Computing the challenge validity bits.* If $A[z]$ receives an input that is structured according to the following form $T[z] = (T'[z], s^*_1[z], \ldots, s^*_p[z])$ where for the prefix $T'[z]$ we have $T'[z] = (\mathsf{pk}[z], \mathsf{cert}[z], r[z], (c_1, m_1)_{3\kappa}[z], (w'_1)_{3\kappa}[z], \ldots, (c_y, m_y)_{3\kappa}[z], (w'_y)_{3\kappa}[z])$, the simulated attacker $A[z]$ first checks whether the prefix is plausible in the same way as before. If not, M aborts the simulation of $A[z]$. Next, the execution state of B is stored by M as $state^*[z]$. Now, $A[z]$ will rewind B to obtain the validity bits for the challenge statements. For each challenge statement $s^*_i[z] = (c^*_i[z], m^*_i[z])$ with $i \in [p]$, $A[z]$ executes the following procedure.

   3.i.1. M iterates through all possible states $\mathsf{state}_n[z]$ for $n \in [y]$.

      3.i.n.1. M rewinds B back to $\mathsf{state}_n[z]$. Similar to the first run, M invokes the algorithm $((c'_i, m'_i)_{3\kappa}[z], \pi'_i[z]) \leftarrow \mathsf{VectorSample}(\mathsf{pk}[z], c^*_i[z], m^*_i[z])$ but now uses the challenge $(c^*_i[z], m^*_i[z])$ as input to $\mathsf{VectorSample}$. Attacker $A[z]$ sends the query $(c'_i, m'_i)_{3\kappa}[z]$ to B.

      3.i.n.2. We abort the execution at $\mathsf{state}_n[z]$, and go to Step 3.i.$n+1$.1. if $n < y$ and either one of the following cases occurs: i) B sends an external query to its tICA challenger before it sends its response $(w'_i)_{3\kappa}[z]$ (to query $(c'_i, m'_i)_{3\kappa}[z]$) to $A[z]$, or ii) B requires a response to a challenge query of another attacker instance $z' \neq z$ before it sends its response to $A[z]$, or iii) B's response $(w'_i)_{3\kappa}[z]$ will make $\mathsf{VectorTest}(\mathsf{pk}[z], (c'_i, m'_i)_{3\kappa}[z], (w'_i)_{3\kappa}[z])$ output 0 (which means that one of the validity bits send by B is invalid). If either of the three cases occurs and $n = y$, we return to Step 3.i.1. and repeat the entire computation using fresh randomness.[7]

      3.i.n.3. We set w^*_i to be the j-th component of $(w'_i)_{3\kappa}[z]$ where we set $j = \pi'_i(2\kappa+1)[z]$. If $i < p$, we proceed to Step 3.$i+1$.n.1. If $i = p$, then we go to step 4.

4. M loads state $state^*[z]$ and sends the validity bits $w^*_1, \ldots, w^*_p$ to B.

5. If at any point, B makes a query that is directed to its tICA challenger, M relays this query to the tICA challenger and the response back to B.

[7] This is very similar to the check before rewinding except that now we do not know what the third validity bit is.

6.3 Proof of Theorem 2

We now provide a series of lemmas that will ultimately be used to prove Theorem 5. For these lemmas, we consider A to be the ideal attacker as described in Section 6.1, and M to be the meta-reduction as described in Section 6.2. We let B be a rewinding reduction, reducing the 3κ-wIND-PCA security of some SHPKE scheme to the security of some tICA. Assume that when given access to the successful attacker A, B breaks the underlying tICA with probability ϵ.

First, we apply the splitting lemma to determine the probability that the randomness r_B given to B admits many different runs of A that make B accept. In other words, r_B is super-good, using the terminology of the Splitting Lemma. To this end let us first introduce configurations. For attacker instance z, a configuration is defined to be a sequence of queries $(q_1[z], ..., q_i[z])$ for $i \in [y]$. For all attacker instances $1, ..., u$, a configuration has the following form $((q_1[1], ..., q_{i_1}[1]), ..., (q_1[u], ..., q_{i_u}[u]))$, i.e. in attacker instance $u' \in [u]$, queries $q_1[u'], ..., q_{i_{u'}}[u']$ have been sent by $A[u']$. Note that for at least one of the attacker instances, we must have that $i_j = y$ for $j \in [u]$ as otherwise, A would never have to solve any challenge statement. (In this case, B could break the tICA without the help of A contradicting the security of the tICA.)

Lemma 3. *With probability $\epsilon/2$, the randomness r_B given to B, will make B accept a total of $\epsilon/2$ of all configurations.*

Proof. For simplicity of notation, we denote a 3κ-PCA query by $q_i = (c_i, m_i)_{3\kappa}$. Let us first provide an argument to show that B accepts many configurations. We define the set Q' to be the set containing all configurations that could potentially get accepted by B before breaking the tICA.

Now we apply the splitting lemma on set Q', to show that the randomness r_B makes B accept at least a fraction $\epsilon' = \epsilon/2$ of the configurations $q' \in Q'$. We set $X = D$, the set containing the possible randomness of B, and set $Y = Q'$. We let G be the set of all pairs $(r_B, q) \in D \times Q'$ that lets B break the tICA. As we assume that B breaks the underlying tICA with probability ϵ, we have that $|G|/|D \times Q'| \geq \epsilon$. Thus, the randomness r_B is considered to be super-good (according to our terminology for the splitting lemma). □

Therefore, in the next proofs (of Lemmas 4 – 8), we assume that B uses fixed randomness r' and assume that it is always super-good. Moreover, we will accordingly consider B to have success probability $\epsilon' = \epsilon/2$ from now on. However, for simplicity we make this implicit. In all cases, these adaptions only account for a non-negligible decrease in success probability or a polynomial increase in running time overall.

Lemma 4. *For our construction of meta-reduction M, there always exists a useful rewinding spot with overwhelming probability.*

Proof. We show, using Hoeffding's inequality (see Lemma 2), that there exists a useful rewinding spot with overwhelming probability. In particular, we apply Hoeffding's inequality to show that there exists a rewinding spot such that i) the query has the same validity bit as the challenge statement, ii) the reduction does not start a new attacker instance or perform an external query, and iii) the reduction sends a correct response. Let us consider conditions i) and ii) first. Then, we argue why case iii) must occur as well for some spot. More concretely, we show now that there are no fewer than $t + u$ and no more than $y - (t + u)$ queries in the first run that share a validity bit with the challenge statement. This allows cases i) and ii) to occur for at least one slot, concluding the existence of a good rewinding spot.

Let X_i be the discrete boolean event such that *the i-th query shares the validity bit with the challenge statement.* Let $S_y = \sum_1^y X_i$. We determine the probability of the following events: i)

S_y is less than $t+u$ or ii) S_y is more than $y-(t+u)$. Observe that $E[S_y] = \frac{y}{2} = \kappa + t + u$. We obtain

$$\mathsf{Pr}[S_y < t+u] = \mathsf{Pr}[E[S_y] - S_y > \kappa].$$

For the second case, we use that $y = 2t + 2\kappa + 2u$ to obtain

$$\mathsf{Pr}[S_y > y-(t+u)] = \mathsf{Pr}[S_y - E[S_y] > \kappa].$$

We combine these probabilities and apply Hoeffding's inequality to obtain that

$$\mathsf{Pr}[|S_y - E[S_y]| > \kappa] \leq 2e^{-2\kappa^2/(2t+2\kappa+2u)}.$$

For $t+u < O(\kappa^2)$, we have that this probability is negligibly small in κ. Therefore, with overwhelming probability, in the first run, there are more than $t+u$ or fewer than $y-(t+u)$ queries that share the same validity bit. Thus, there exists at least one query in the first run that shares the validity bit with the challenge statement, for which B did not invoke an external query nor initiate a new attacker instance. As the queries made by $A[z]$ during rewinding are distributed exactly like those of the ideal attacker (except with statistically low error δ), we conclude that this spot is a good candidate for a useful rewinding spot. The error probability δ reflects the fact that in the rewinding phase VectorSample is *given* the challenge as input whereas the ideal attacker generates the input to VectorSample by calling RSample. However, RSample has failure probability δ.

Lastly, we analyze the probability that B can send an invalid response in this candidate spot (during the rewinding phase) that A will not notice. (Otherwise, A simply aborts.) The meta-reduction can use VectorTest to verify whether the response B is valid or not. By the construction of VectorTest, B has to guess which of the κ statements (c_i, m_i) (out of 2κ, as there are 2κ statements in a 3κ-PCA query sharing a validity bit) correspond to the challenge statement. It should then flip all validity bits for these statements. The probability of B succeeding in this is given by $\binom{2\kappa}{\kappa}^{-1} \leq 2^{-\kappa}$, which is negligible in κ. This probability is still negligible taken into account that B may rewind $A[z]$. This concludes that a useful rewinding slot exists. □

Now let us analyze the running time of the meta-reduction.

Lemma 5. *M has expected polynomial running time (in κ).*

Proof. We show that the meta-reduction M has an expected polynomial running time. Most steps are efficient by construction. We have to analyze the running time of the rewinding phase, which is dominated by steps 3.1.1.1 – 3.$p.y$.3. We show that the expected number of jumps within these steps is polynomial. For an attacker instance z, we define $E_{i,n}$ as *the event, that after sending a 3κ-PCA query to B in state_n, the meta-reduction receives a response w_i^* while B did not perform an external query, nor did it send a challenge statement for another $z' \neq z$ instance, nor did* VectorTest *output 0 for the response of B.* By Lemma 3, it follows that B accepts an arbitrary configuration with probability ϵ'. Let z be the attacker instance. Let $(q_1[z], ..., q_y[z])$ be the configuration that has been sent in the first run. Then, we know that this configuration is accepted by B. Otherwise, the rewinding procedure would not have been started. Any individual statement q_i will be responded to with probability at least ϵ'. Assume that we load the internal state state_n of B in order to find the validity bit of w_i^* for $i \in [p]$. Then, the configuration will be of the following form $q_1, ..., q_{n-1}, q_i'$ where q_i' is a randomization of challenge statement s_i^*. This configuration is distributed in the same way as the one from the ideal attacker A, except with probability δ. This is because the ideal attacker uses RSample to generate the input to VectorSample. But RSample has success probability $1-\delta$. Since we know that among the

y queries there always exists a useful rewinding spot, we have $\mathsf{Pr}[E_{i,n}] \geq (\epsilon' - \delta)/y$. Of course, the same lower bound holds if we consider the event E_i that for at least one of the possible rewinding spots, B will accept the configuration, i.e. $\mathsf{Pr}[E_i] = \mathsf{Pr}[E_{i,1} \vee ... \vee E_{i,y}] \geq (\epsilon' - \delta)/y$. Therefore, the running time of the meta-reduction for finding a validity bit for challenge s_i^* is $x_i = O(1/\mathsf{Pr}[E_i])$, which is polynomial. As we can verify the responses of B immediately after we receive them (using VectorTest), we repeat this argument p times, once for each challenge statement. Therefore, the expected number of jumps that are made to answer all challenge statements is polynomial as well. This shows that the expected running time for attacker instance z is polynomial. Observe that the argument in the proof is independent of the choice of z. Therefore, the overall running time of M is expected to be polynomial time. □

The next two lemmas focus on the success probability of the meta-reduction M when rewinding the reduction B.

Lemma 6. *If M does not abort early on, it outputs validity bits $w_1^*, \ldots, w_p^*$ that are valid with a probability that is non-negligibly better than $1/2^p$.*

Proof. In Lemma 4, we have shown that there exists a useful rewinding spot. In Lemma 5, we have shown that the running time of M is polynomial. Therefore, by our construction, if M and B do not abort the security game, then M outputs a sequence of validity bits $w_1^*, ..., w_p^*$.

The fact that the validity bits $w_1^*, ..., w_p^*$ output by M are valid with overwhelming probability is a direct consequence of Lemma 4 and the fact that the validity bits are not changed when applying Randomize. Thus the response received by M is valid with probably non-negligibly better than $1/2^p$. □

We apply Lemmas 4 and 6 to show that B is not able to distinguish the ideal attacker A from the meta-reduction M with probability statistically close to 1.

Lemma 7. *The reduction B is not able to distinguish A and M with probability statistically close to one.*

Proof. We show that reduction B cannot distinguish the ideal attacker A and meta-reduction M with a probability statistically close to 1. In the first run, both A and M behave similarly. Reduction B may only spot a difference in behaviour if M fails to extract valid witnesses during rewinding. This may either be because i) B delivered an invalid witness without M noticing, or ii) B has aborted during the second run. For the first case, we have seen that this probability is at least $1/2^\kappa$ (see Lemma 4), which is statistically small in κ. For the second case, we observe that a query made by M is distributed statistically close to a query by the ideal attacker A. Each 3κ-PCA query consists of 3κ individual statements. However, the only difference in the way they are generated is the way in which the input to VectorSample is generated. All other computations remain the same. Thus, for each rewinding, the queries are equally distributed except with probability δ so B cannot distinguish A and M with a probability statistically close to 1. □

Finally, we analyze the number of queries that M has to make to the ICA challenger throughout a run.

Lemma 8. *M will make at most t queries to the ICA challenger.*

Proof. By our construction, meta-reduction M simply relays queries from the reduction to tICA without allowing the reduction to execute more of such queries. By carefully searching for useful rewinding spots, M actively prevents B from making additional queries because of the rewinding. Therefore, the meta-reduction does not make more than t queries to the tICA challenger. □

Using these lemmas, we can finally prove Theorem 2.

Proof of Theorem 2. Using the previous lemmas we have established that i) M runs in expected polynomial time, ii) B cannot distinguish A from M, and iii) M outputs the correct responses to the challenges for every attacker instance. However, the combination of B and M efficiently break the security of the tICA. So, under the assumption that tICA is secure, B cannot exist. □

Since $3k$-wIND-PCA security is weaker than wIND-PCA security for the same overall number of queried ciphertext/plaintext pairs and as the attacker in the 3κ-wIND-PCA game queries $3\kappa(2t + 2\kappa + 2u)$ such pairs, we immediately obtain the following result.

Theorem 3. *Let κ be the security parameter. Let p, the number of challenge statements that the attacker must solve in the wIND-PCA1 security game, be polynomial. Then, there exists no PPT reduction B that reduces the wIND-PCA1 security of a certified semi-homomorphic PKE with $3\kappa(2t+2\kappa+2u)$ adaptive PCA queries to the security of any t-interactive complexity assumption, while creating no more than u attacker instances, for $t + u < O(\kappa^2)$.*

6.4 Applications of Theorem 3

We can now immediately present some important applications of Theorem 3. We begin with a corollary stating that ElGamal PKE cannot be proven IND-PCA1 secure. Next, we present a similar result for Paillier PKE.

Corollary 2. *Let κ be the security parameter. Let p, the number of challenge statements that the attacker must solve in the IND-PCA1 security game, be polynomial. Then, there exists no PPT reduction B that reduces the IND-PCA1 security of ElGamal PKE with $3\kappa(2t + 2\kappa + 2u)$ adaptive PCA queries to the security of any t-interactive complexity assumption, while creating no more than u attacker instances for $t + u < O(\kappa^2)$.*

Corollary 3. *Let κ be the security parameter. Let p, the number of challenge statements that the attacker must solve in the IND-PCA1 security game, be polynomial. Then, there exists no PPT reduction B that reduces the IND-PCA1 security of Paillier PKE with $3\kappa(2t + 2\kappa + 2u)$ adaptive PCA queries to the security of any t-interactive complexity assumption, while creating no more than u attacker instances for $t + u < O(\kappa^2)$.*

6.5 Impact, Interpretation, and Limitations

We present new impossibility results for the IND-PCA security of a class of well-known semi-homomorphic cryptographic schemes under a very liberal notion of reduction. Switching perspectives, our results more generally provide a simple method to identify whether it is impossible to prove a semi-homomorphic PKE scheme secure under IND-PCA attacks (or any other strictly stronger notion). Of course, all our results transfer to schemes that additionally support the homomorphic evaluation of a second operation, making them a valuable tool in the design of FHE schemes. At the same time, our impossibility results require several restrictive assumptions to hold. Crucially, these assumptions imply the existence of reliable membership tests for valid public keys and ciphertexts that only work on public parameters. In contrast to this, existing schemes (even without homomorphic properties) that provably provide stronger security guarantees (e.g. IND-CCA2), usually do not provide such algorithms. On the contrary, their security arguments often rely on the very fact that it is hard to separate valid from invalid ciphertexts [12].

Finally, we observe that since the recent notion of indistinguishability against verified chosen ciphertext-attack (vCCA) by Manulis and Nguyen implies IND-CCA1 security (and thus also

IND-PCA1 security) [36], our results are meaningful as well. Interpreted accordingly, our result shows that the class of schemes we consider in this work cannot provide IND-vCCA security.

It is well known that when using programmable random oracles (pROs), meta-reduction-based impossibility results can be sidestepped [26]. Essentially, this is due to the fact that pROs allow the construction of cryptographic primitives that are non-committing for the reduction. Transferred to our setting, such a reduction could only decide on-the-fly whether a ciphertext/plaintext pair has validity bit 0 or 1 by programming the pRO correspondingly. Problematically, the challenges output by the reduction could have undefined validity bits. Furthermore, and in a similar way, our results do not immediately extend to general setup routines that establish non-certified common reference strings.

In Appendix B, we also provide an impossibility result for the much more restricted class of simple reductions. Although the considered type of reductions is restricted, it provides better quantitative bounds on the number of IND-PCA queries for otherwise equal parameters, applying already in case the attacker only makes $2t + 2\kappa$ PCA-queries as opposed to $3\kappa(2t + 2\kappa + 2u)$ queries. It is unclear whether these bounds are optimal.

References

[1] Michel Abdalla, Fabrice Benhamouda, and David Pointcheval. Public-key encryption indistinguishable under plaintext-checkable attacks. In Jonathan Katz, editor, *PKC 2015*, volume 9020 of *LNCS*, pages 332–352, Gaithersburg, MD, USA, March 30 – April 1, 2015. Springer, Heidelberg, Germany.

[2] Adi Akavia, Craig Gentry, Shai Halevi, and Margarita Vald. Achievable CCA2 relaxation for homomorphic encryption. In Eike Kiltz and Vinod Vaikuntanathan, editors, *TCC 2022, Part II*, volume 13748 of *LNCS*, pages 70–99, Chicago, IL, USA, November 7–10, 2022. Springer, Heidelberg, Germany.

[3] Christoph Bader, Tibor Jager, Yong Li, and Sven Schäge. On the impossibility of tight cryptographic reductions. In Marc Fischlin and Jean-Sébastien Coron, editors, *EUROCRYPT 2016, Part II*, volume 9666 of *LNCS*, pages 273–304, Vienna, Austria, May 8–12, 2016. Springer, Heidelberg, Germany.

[4] Foteini Baldimtsi and Anna Lysyanskaya. On the security of one-witness blind signature schemes. In Kazue Sako and Palash Sarkar, editors, *ASIACRYPT 2013, Part II*, volume 8270 of *LNCS*, pages 82–99, Bengalore, India, December 1–5, 2013. Springer, Heidelberg, Germany.

[5] Mihir Bellare, Thomas Ristenpart, and Stefano Tessaro. Multi-instance security and its application to password-based cryptography. In Reihaneh Safavi-Naini and Ran Canetti, editors, *CRYPTO 2012*, volume 7417 of *LNCS*, pages 312–329, Santa Barbara, CA, USA, August 19–23, 2012. Springer, Heidelberg, Germany.

[6] Dan Boneh, Xavier Boyen, and Hovav Shacham. Short group signatures. In Matthew Franklin, editor, *CRYPTO 2004*, volume 3152 of *LNCS*, pages 41–55, Santa Barbara, CA, USA, August 15–19, 2004. Springer, Heidelberg, Germany.

[7] Dan Boneh, Gil Segev, and Brent Waters. Targeted malleability: homomorphic encryption for restricted computations. In Shafi Goldwasser, editor, *Innovations in Theoretical Computer Science 2012, Cambridge, MA, USA, January 8-10, 2012*, pages 350–366. ACM, 2012.

[8] Dan Boneh and Ramarathnam Venkatesan. Breaking RSA may not be equivalent to factoring. In Kaisa Nyberg, editor, *EUROCRYPT'98*, volume 1403 of *LNCS*, pages 59–71, Espoo, Finland, May 31 – June 4, 1998. Springer, Heidelberg, Germany.

[9] Ran Canetti, Oded Goldreich, Shafi Goldwasser, and Silvio Micali. Resettable zero-knowledge (extended abstract). In *32nd ACM STOC*, pages 235–244, Portland, OR, USA, May 21–23, 2000. ACM Press.

[10] Ran Canetti, Huijia Lin, and Rafael Pass. Adaptive hardness and composable security in the plain model from standard assumptions. In *51st FOCS*, pages 541–550, Las Vegas, NV, USA, October 23–26, 2010. IEEE Computer Society Press.

[11] Jean-Sébastien Coron. Optimal security proofs for PSS and other signature schemes. In Lars R. Knudsen, editor, *EUROCRYPT 2002*, volume 2332 of *LNCS*, pages 272–287, Amsterdam, The Netherlands, April 28 – May 2, 2002. Springer, Heidelberg, Germany.

[12] Ronald Cramer and Victor Shoup. A practical public key cryptosystem provably secure against adaptive chosen ciphertext attack. In Hugo Krawczyk, editor, *CRYPTO'98*, volume 1462 of *LNCS*, pages 13–25, Santa Barbara, CA, USA, August 23–27, 1998. Springer, Heidelberg, Germany.

[13] Ivan Damgård. Towards practical public key systems secure against chosen ciphertext attacks. In Joan Feigenbaum, editor, *Advances in Cryptology - CRYPTO '91, 11th Annual International Cryptology Conference, Santa Barbara, California, USA, August 11-15, 1991, Proceedings*, volume 576 of *Lecture Notes in Computer Science*, pages 445–456. Springer, 1991.

[14] Ivan Damgård and Mats Jurik. A generalisation, a simplification and some applications of Paillier's probabilistic public-key system. In Kwangjo Kim, editor, *PKC 2001*, volume 1992 of *LNCS*, pages 119–136, Cheju Island, South Korea, February 13–15, 2001. Springer, Heidelberg, Germany.

[15] Yi Deng, Vipul Goyal, and Amit Sahai. Resolving the simultaneous resettability conjecture and a new non-black-box simulation strategy. In *50th FOCS*, pages 251–260, Atlanta, GA, USA, October 25–27, 2009. IEEE Computer Society Press.

[16] Yevgeniy Dodis, Roberto Oliveira, and Krzysztof Pietrzak. On the generic insecurity of the full domain hash. In Victor Shoup, editor, *CRYPTO 2005*, volume 3621 of *LNCS*, pages 449–466, Santa Barbara, CA, USA, August 14–18, 2005. Springer, Heidelberg, Germany.

[17] Yevgeniy Dodis and Leonid Reyzin. On the power of claw-free permutations. In Stelvio Cimato, Clemente Galdi, and Giuseppe Persiano, editors, *SCN 02*, volume 2576 of *LNCS*, pages 55–73, Amalfi, Italy, September 12–13, 2003. Springer, Heidelberg, Germany.

[18] Alex Escala, Gottfried Herold, Eike Kiltz, Carla Ràfols, and Jorge Villar. An algebraic framework for Diffie-Hellman assumptions. In Ran Canetti and Juan A. Garay, editors, *CRYPTO 2013, Part II*, volume 8043 of *LNCS*, pages 129–147, Santa Barbara, CA, USA, August 18–22, 2013. Springer, Heidelberg, Germany.

[19] Prastudy Fauzi, Martha Norberg Hovd, and Håvard Raddum. On the IND-CCA1 security of FHE schemes. *Cryptogr.*, 6(1):13, 2022.

[20] Marc Fischlin and Nils Fleischhacker. Limitations of the meta-reduction technique: The case of Schnorr signatures. In Thomas Johansson and Phong Q. Nguyen, editors, *EUROCRYPT 2013*, volume 7881 of *LNCS*, pages 444–460, Athens, Greece, May 26–30, 2013. Springer, Heidelberg, Germany.

[21] Nils Fleischhacker, Tibor Jager, and Dominique Schröder. On tight security proofs for Schnorr signatures. In Palash Sarkar and Tetsu Iwata, editors, *ASIACRYPT 2014, Part I*, volume 8873 of *LNCS*, pages 512–531, Kaoshiung, Taiwan, R.O.C., December 7–11, 2014. Springer, Heidelberg, Germany.

[22] Sanjam Garg, Raghav Bhaskar, and Satyanarayana V. Lokam. Improved bounds on security reductions for discrete log based signatures. In David Wagner, editor, *CRYPTO 2008*, volume 5157 of *LNCS*, pages 93–107, Santa Barbara, CA, USA, August 17–21, 2008. Springer, Heidelberg, Germany.

[23] Rosario Gennaro, Steven Goldfeder, and Arvind Narayanan. Threshold-optimal DSA/ECDSA signatures and an application to bitcoin wallet security. In Mark Manulis, Ahmad-Reza Sadeghi, and Steve Schneider, editors, *ACNS 16*, volume 9696 of *LNCS*, pages 156–174, Guildford, UK, June 19–22, 2016. Springer, Heidelberg, Germany.

[24] Rosario Gennaro and Yehuda Lindell. A framework for password-based authenticated key exchange. In Eli Biham, editor, *EUROCRYPT 2003*, volume 2656 of *LNCS*, pages 524–543, Warsaw, Poland, May 4–8, 2003. Springer, Heidelberg, Germany. `https://eprint.iacr.org/2003/032.ps.gz`.

[25] Adam Groce and Jonathan Katz. A new framework for efficient password-based authenticated key exchange. In Ehab Al-Shaer, Angelos D. Keromytis, and Vitaly Shmatikov, editors, *ACM CCS 2010*, pages 516–525, Chicago, Illinois, USA, October 4–8, 2010. ACM Press.

[26] Fuchun Guo, Rongmao Chen, Willy Susilo, Jianchang Lai, Guomin Yang, and Yi Mu. Optimal security reductions for unique signatures: Bypassing impossibilities with a counterexample. In Jonathan Katz and Hovav Shacham, editors, *CRYPTO 2017, Part II*, volume 10402 of *LNCS*, pages 517–547, Santa Barbara, CA, USA, August 20–24, 2017. Springer, Heidelberg, Germany.

[27] Dennis Hofheinz, Tibor Jager, and Edward Knapp. Waters signatures with optimal security reduction. In Marc Fischlin, Johannes Buchmann, and Mark Manulis, editors, *PKC 2012*, volume 7293 of *LNCS*, pages 66–83, Darmstadt, Germany, May 21–23, 2012. Springer, Heidelberg, Germany.

[28] Saqib A. Kakvi and Eike Kiltz. Optimal security proofs for full domain hash, revisited. In David Pointcheval and Thomas Johansson, editors, *EUROCRYPT 2012*, volume 7237 of *LNCS*, pages 537–553, Cambridge, UK, April 15–19, 2012. Springer, Heidelberg, Germany.

[29] Jonathan Katz and Vinod Vaikuntanathan. Round-optimal password-based authenticated key exchange. In Yuval Ishai, editor, *TCC 2011*, volume 6597 of *LNCS*, pages 293–310, Providence, RI, USA, March 28–30, 2011. Springer, Heidelberg, Germany.

[30] Gregor Leander and Andy Rupp. On the equivalence of RSA and factoring regarding generic ring algorithms. In Xuejia Lai and Kefei Chen, editors, *ASIACRYPT 2006*, volume 4284 of *LNCS*, pages 241–251, Shanghai, China, December 3–7, 2006. Springer, Heidelberg, Germany.

[31] Allison B. Lewko and Brent Waters. Why proving HIBE systems secure is difficult. In Phong Q. Nguyen and Elisabeth Oswald, editors, *EUROCRYPT 2014*, volume 8441 of *LNCS*, pages 58–76, Copenhagen, Denmark, May 11–15, 2014. Springer, Heidelberg, Germany.

[32] Benoît Libert. Leveraging small message spaces for CCA1 security in additively homomorphic and bgn-type encryption. In Serge Fehr and Pierre-Alain Fouque, editors, *Advances in Cryptology - EUROCRYPT 2025 - 44th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Madrid, Spain, May 4-8, 2025, Proceedings, Part II*, volume 15602 of *Lecture Notes in Computer Science*, pages 34–63. Springer, 2025.

[33] Yehuda Lindell. Fast secure two-party ECDSA signing. In Jonathan Katz and Hovav Shacham, editors, *CRYPTO 2017, Part II*, volume 10402 of *LNCS*, pages 613–644, Santa Barbara, CA, USA, August 20–24, 2017. Springer, Heidelberg, Germany.

[34] Helger Lipmaa. On the CCA1-security of Elgamal and Damgård's Elgamal. In Xuejia Lai, Moti Yung, and Dongdai Lin, editors, *Information Security and Cryptology - 6th International Conference, Inscrypt 2010, Shanghai, China, October 20-24, 2010, Revised Selected Papers*, volume 6584 of *Lecture Notes in Computer Science*, pages 18–35. Springer, 2010.

[35] Jake Loftus, Alexander May, Nigel P. Smart, and Frederik Vercauteren. On CCA-secure somewhat homomorphic encryption. In Ali Miri and Serge Vaudenay, editors, *Selected Areas in Cryptography - 18th International Workshop, SAC 2011, Toronto, ON, Canada, August 11-12, 2011, Revised Selected Papers*, volume 7118 of *Lecture Notes in Computer Science*, pages 55–72. Springer, 2011.

[36] Mark Manulis and Jérôme Nguyen. Fully homomorphic encryption beyond IND-CCA1 security: Integrity through verifiability. In Marc Joye and Gregor Leander, editors, *Advances in Cryptology - EUROCRYPT 2024 - 43rd Annual International Conference on the Theory and Applications of Cryptographic Techniques, Zurich, Switzerland, May 26-30, 2024, Proceedings, Part II*, volume 14652 of *Lecture Notes in Computer Science*, pages 63–93. Springer, 2024.

[37] Ueli M. Maurer. Abstract models of computation in cryptography. In Nigel P. Smart, editor, *Cryptography and Coding, 10th IMA International Conference, Cirencester, UK, December 19-21, 2005, Proceedings*, volume 3796 of *Lecture Notes in Computer Science*, pages 1–12. Springer, 2005.

[38] Andrew Morgan and Rafael Pass. On the security loss of unique signatures. In Amos Beimel and Stefan Dziembowski, editors, *TCC 2018, Part I*, volume 11239 of *LNCS*, pages 507–536, Panaji, India, November 11–14, 2018. Springer, Heidelberg, Germany.

[39] Andrew Morgan, Rafael Pass, and Elaine Shi. On the adaptive security of MACs and PRFs. In Shiho Moriai and Huaxiong Wang, editors, *ASIACRYPT 2020, Part I*, volume 12491 of *LNCS*, pages 724–753, Daejeon, South Korea, December 7–11, 2020. Springer, Heidelberg, Germany.

[40] Pascal Paillier. Public-key cryptosystems based on composite degree residuosity classes. In Jacques Stern, editor, *EUROCRYPT'99*, volume 1592 of *LNCS*, pages 223–238, Prague, Czech Republic, May 2–6, 1999. Springer, Heidelberg, Germany.

[41] Pascal Paillier and Damien Vergnaud. Discrete-log-based signatures may not be equivalent to discrete log. In Bimal K. Roy, editor, *ASIACRYPT 2005*, volume 3788 of *LNCS*, pages 1–20, Chennai, India, December 4–8, 2005. Springer, Heidelberg, Germany.

[42] Pascal Paillier and Jorge L. Villar. Trading one-wayness against chosen-ciphertext security in factoring-based encryption. In Xuejia Lai and Kefei Chen, editors, *ASIACRYPT 2006*, volume 4284 of *LNCS*, pages 252–266, Shanghai, China, December 3–7, 2006. Springer, Heidelberg, Germany.

[43] Rafael Pass. Limits of provable security from standard assumptions. In Lance Fortnow and Salil P. Vadhan, editors, *43rd ACM STOC*, pages 109–118, San Jose, CA, USA, June 6–8, 2011. ACM Press.

[44] Rafael Pass and Muthuramakrishnan Venkitasubramaniam. On constant-round concurrent zero-knowledge. In Ran Canetti, editor, *TCC 2008*, volume 4948 of *LNCS*, pages 553–570, San Francisco, CA, USA, March 19–21, 2008. Springer, Heidelberg, Germany.

[45] David Pointcheval and Jacques Stern. Security proofs for signature schemes. In Ueli M. Maurer, editor, *EUROCRYPT'96*, volume 1070 of *LNCS*, pages 387–398, Saragossa, Spain, May 12–16, 1996. Springer, Heidelberg, Germany.

[46] Manoj Prabhakaran and Mike Rosulek. Homomorphic encryption with CCA security. In Luca Aceto, Ivan Damgård, Leslie Ann Goldberg, Magnús M. Halldórsson, Anna Ingólfsdóttir, and Igor Walukiewicz, editors, *ICALP 2008, Part II*, volume 5126 of *LNCS*, pages 667–678, Reykjavik, Iceland, July 7–11, 2008. Springer, Heidelberg, Germany.

[47] Ransom Richardson and Joe Kilian. On the concurrent composition of zero-knowledge proofs. In Jacques Stern, editor, *EUROCRYPT'99*, volume 1592 of *LNCS*, pages 415–431, Prague, Czech Republic, May 2–6, 1999. Springer, Heidelberg, Germany.

[48] Sven Schäge. New limits of provable security and applications to ElGamal encryption. In Marc Joye and Gregor Leander, editors, *Advances in Cryptology - EUROCRYPT 2024 - 43rd Annual International Conference on the Theory and Applications of Cryptographic Techniques, Zurich, Switzerland, May 26-30, 2024, Proceedings, Part IV*, volume 14654 of *Lecture Notes in Computer Science*, pages 255–285. Springer, 2024.

[49] Yannick Seurin. On the exact security of Schnorr-type signatures in the random oracle model. In David Pointcheval and Thomas Johansson, editors, *EUROCRYPT 2012*, volume 7237 of *LNCS*, pages 554–571, Cambridge, UK, April 15–19, 2012. Springer, Heidelberg, Germany.

[50] Victor Shoup. Lower bounds for discrete logarithms and related problems. In Walter Fumy, editor, *EUROCRYPT'97*, volume 1233 of *LNCS*, pages 256–266, Konstanz, Germany, May 11–15, 1997. Springer, Heidelberg, Germany.

A ElGamal PKE

As our result will be directly applied to the ElGamal PKE, we will provide a formal description.

1. $\mathsf{PKE.KGen}(1^\kappa)$. The $\mathsf{PKE.KGen}$ algorithm takes as input κ and outputs an arbitrary prime q and generator g. The bit length of q should be polynomial in κ. Next to that, for g it should hold that $\langle g \rangle = G$. Next, it draws an arbitrary $\mathsf{sk} = x \in \mathbb{Z}_q$ and outputs the corresponding public key $\mathsf{pk} = (g, h = g^x, q)$.

2. $\mathsf{PKE.Enc}(\mathsf{pk}, m)$. The $\mathsf{PKE.Enc}$ algorithm takes as input a plaintext m and the public key pk. It also draws some randomness $r \in \mathbb{Z}_q$ uniformly random. It outputs a ciphertext $c = (c_1, c_2) = (g^r, m \cdot h^r) \in G^2$.

3. $\mathsf{PKE.Dec}(\mathsf{sk}, c)$. The $\mathsf{PKE.Dec}$ algorithm takes as input ciphertext $c = (c_1, c_2) \in G^2$ and outputs the corresponding plaintext $m = c_2/c_1^x$.

One could also have a precomputation phase in which q, G, g are determined. This does not influence our result for ElGamal PKE. We remark that all values available to the attacker A, ensure that every ciphertext c has a unique plaintext.

B Impossibility of wIND-PCA1 Security for Simple Reductions

In this section, we present our impossibility result for simple reductions. The necessary restrictions on simple reductions, in contrast to general reductions, are given first.

B.1 Simple Reductions

A simple reduction B only has black-box access to the attacker. The reduction may only execute a single instance of the attacker. Lastly, the reduction is not allowed to rewind the attacker. These restrictions allow us to obtain better bounds than in our result for general reductions. In Section 6, we establish the result for general (arbitrary) reductions. It is important to note that simple reductions are widely used in cryptography.

B.2 Main Result for Simple Reductions

Theorem 4 is our main result for simple reductions.

Theorem 4. *Let* PKE *be a SHPKE scheme with security parameter* κ*. Let* p*, the number of challenge statements that the attacker must solve in the wIND-PCA1 security game, be constant. Then, there exists no simple PPT reduction* B *that reduces the security of the wIND-PCA1 security game with* $2t + 2\kappa$ *adaptive PCA queries to the security of any* t*-interactive complexity assumption* ICA*.*

B.3 Sketch of the Proof

We start by providing some intuition. We assume the existence of an ideal attacker A with unbounded computational resources. Next, we show how we can efficiently simulate the ideal attacker A. This is done via an efficient meta-reduction M. The main strategy of the meta-reduction is to rewind the reduction B to break the security game, whereas the ideal attacker may

use any (inefficient) method to break the security game. The crucial step is that the reduction should not be able to distinguish the ideal attacker A and the meta-reduction M.

Intuitively, the parameters chosen ensure that the number of useful rewinding spots for the meta-reduction is sufficiently high. A rewinding spot is a state in the learning phase at which the attacker can execute a query to the reduction. A rewinding spot is useful if the reduction responds to the query sent correctly without having executed an external query to its challenger. The big challenge is that, due to our security game being based on a decision problem, the reduction may check if the query (statement) made has the same validity bit as the challenge statement. By our construction of the ideal attacker and meta-reduction, on average, half of the queries that are sent in the first run have validity bit $w = 1$, and thus the other half will have validity bit $w = 0$. For security parameter κ, we consider an attacker that makes $2t + 2\kappa$ PCA queries. We assume the reduction performs at most t external queries. This ensures that the reduction is not able to perform external queries whenever the attacker sends a query that shares the validity bit with the challenger. More concretely, we use Hoeffding's inequality (see Lemma 2) to show that these parameters ensure that there will be a spot in which i) the reduction does not perform an external query, and ii) the query sent by the attacker has the same validity bit as the one in B's challenge. Technically, we determine the probability that the least common validity bit in the first run occurs at least t times, as the reduction may adapt the choice of the challenge based on the learning phase.

It is enough that the ideal attacker makes the reduction behave honestly with non-negligible probability. This immediately transfers to the meta-reduction that in the learning phase behaves exactly like the ideal attacker. And since the values that we sent after rewinding will be statistically indistinguishable from those before rewinding (from the viewpoint of the reduction) the reduction will also behave honestly. Our parameters ensure that the fraction of useful rewinding spots is high compared to t. This will be shown in Lemmas 9–14. In the end, the response of the reduction will be used to answer the challenge query.

B.4 The Ideal Attacker A

We present a description of the ideal attacker A for our impossibility result on simple reductions. For simplicity in notation, let $y = 2t + 2\kappa$.

1. On initialization, the challenger sends a public key pk and a proof cert, together with some random coins r, to A. Attacker A checks whether $(\mathsf{pk}, \mathsf{cert})$ are valid and aborts on failure.

2. For $i \in [1; y]$, it performs the following steps.

   2.i.1 The attacker A runs $(c_i, m_i, w_i) \leftarrow \mathsf{RSample}(\mathsf{pk})$ and sends (c_i, m_i) to C.

   2.i.2 The challenger responds with a validity bit w_i'. The attacker A verifies whether $w_i = w_i'$. If this is false, A aborts the security game. If $i = y$ (i.e. after the final query), the challenger also sends the challenge statements $(c_1^*, m_1^*), \ldots, (c_p^*, m_p^*) \in \mathcal{C} \times \mathcal{M}$.

3. The attacker verifies whether all $(c_1^*, m_1^*), \ldots, (c_p^*, m_p^*) \in \mathcal{C} \times \mathcal{M}$ and aborts if not. For each pair (c_i^*, m_i^*), the attacker determines the corresponding validity bit w_i^* using its unbounded computational resources. Then, it sends the validity bits $w_1^*, ..., w_p^*$ to the challenger.

B.5 The Meta-Reduction

In this section, we present the meta-reduction M, that will be able to efficiently simulate the ideal adversary. We assume that M stores the execution state state_i of B before it sends the i-th query, so that it is able to rewind the reduction. For simplicity in notation, we let $y = 2t + 2\kappa$.

0. The meta-reduction M receives the tICA instance c and relays it to B along with random coins $r_B \in D_B$.

1. On initialization, B sends a public key pk and a proof cert, together with some random coins r, to M. If r have never been used before this creates a new instance of the simulated attacker. Meta-reduction M verifies $(\mathsf{pk}, \mathsf{cert})$ and aborts on failure.

2. For $j \in [1; y]$, M performs the following steps.

   2.j.1 First, the meta-reduction M stores the execution state of B as state_i. Next, the meta-reduction M calls $(c_j, m_j, w_j) \leftarrow \mathsf{RSample}(\mathsf{pk})$ and sends (c_j, m_j) to B.

   2.j.2 Reduction B responds with a validity bit w'_j. Then M verifies whether $w_j = w'_j$. If not, the attacker aborts. Otherwise, it continues the execution. If B performs an external query to its tICA challenger, it is relayed by M to its tICA challenger. Any response from the tICA challenger is relayed back to B. If $j = y$ (i.e. after the final query), the challenger also sends the challenge statements $(c_1^*, m_1^*), \ldots, (c_p^*, m_p^*) \in \mathcal{C} \times \mathcal{M}$. Meta-reduction M store this state of B as state^*.

2' M verifies whether all $(c_1^*, m_1^*), \ldots, (c_p^*, m_p^*) \in \mathcal{C} \times \mathcal{M}$ and aborts if not. If M does not abort, it stops the communication with B in the current state and stores the current execution state state^* of B. Starting at $i = 1$, M executes the following procedure for all $i \in [p]$.

   2'.i M iterates through all possible states state_n for $n \in [y]$, starting at $n = 1$.

   2'.i.n.1 M rewinds the reduction back by loading state_n. Instead of sending (c_n, m_n) (which was sent in the first run), M now sends $\mathrm{x}(c'_j, m'_j) \leftarrow \mathsf{Randomize}(\mathsf{pk}, m_j^*, c_j^*)$ as a query to B.

   2'.i.n.2 If B attempts to perform an external query to its tICA challenger (or if it does not respond to the query), we abort the execution and proceed to Step 2'.i.n+1.1. If $n = y$, we return to Step 2'.i. If we receive a response w_j^* and $i < p$, we store it and proceed to Step 2'.$i + 1$. If $i = p$, we proceed to Step 3.

3. The meta-reduction loads state $\mathsf{state}^*$and sends validity bits $(w_1^*, ..., w_p^*)$ to B.

4. Reduction B sends a solution to the tICA challenge.

5. Meta-reduction M relays this solution to the tICA challenger.

B.6 Analysis

We show that the meta-reduction M in Section B.5 can efficiently simulate the ideal attacker A in Section B.4. To show this, we present a sequence of lemmas. For these lemmas, we let A be the ideal attacker as described in Section B.4, and let M be the meta-reduction as described in Section B.5. Assume that B is a simple reduction (see Section B.1) that breaks the underlying security assumption with probability ϵ when given access to a successful attacker A. First, we show that the splitting lemma can be applied to the randomness used by B, see Lemma 9.

Lemma 9. *With probability $\epsilon/2$, the randomness r_B given to B, will make B accept a total of $\epsilon/2$ of all the possible configurations.*

Proof. We apply the splitting lemma (see Lemma 1) by setting X to be the randomness space D. We set $Y = (\mathcal{C} \times \mathcal{M})^y$ to be the space containing all queries and we set G to be the set of all tuples $(r, (c_1, m_1), ..., (c_y, m_y)) \in X \times Y$, that let B break tICA. From this, we can conclude

that with non-negligible probability, at least a fraction $\epsilon' = \epsilon/2$ (which is non-negligible) of all possible queries tuples $((c_1, m_1), \ldots, (c_y, m_y))$ of size y will be accepted by B. For the remaining lemmas, we will assume that such super-good random coins (according to our terminology for the splitting lemma) are given to the reduction at the beginning of the game. This accounts for only a non-negligible decrease in the overall success probability of M. □

In the proofs of Lemmas 10– 14, we assume B to use fixed randomness and assume that this randomness is super-good. Moreover, we will consider B to have a success probability $\epsilon' = \epsilon/2$ from now on. This accounts for only a non-negligible decrease of success probability overall and a polynomial increase of the runtime. Now let us first show that there exists a useful rewinding spot for our meta-reduction.

Lemma 10. *For our construction of meta-reduction M, there always exists a rewinding spot such that, with overwhelming probability, i) the query has the same validity bit as the challenge statement, and ii) the reduction B does not perform an external query.*

Proof. We show that there exists at least one rewinding spot that can be used to extract the validity bit of the challenge statement. We apply Hoeffding's inequality to show that there exists a rewinding spot such that i) the query has the same validity bit as the challenge statement, and ii) the reduction does not perform an external query. More concretely, we show now that there are no less than t and no more than $y - t$ queries in the first run that share a validity bit with the challenge statement. This concludes the existence of a good rewinding spot.

Let X_i be the discrete boolean event such that *the i-th query shares the validity bit with the challenge statement.* Let $S_y = \sum_1^y X_i$. We determine the probability of the following events: i) S_y is less than t or ii) S_y is more than $y - t$. Observe that $E[S_y] = \kappa + t$. We obtain

$$\mathsf{Pr}[S_y < t] = \mathsf{Pr}[E[S_y] - S_y > \kappa].$$

For the second case, we use that $y = 2t + 2\kappa$ to obtain

$$\mathsf{Pr}[S_y > y - t] = \mathsf{Pr}[S_y - E[S_y] > \kappa].$$

We combine these probabilities and apply the Hoeffding inequality to obtain that

$$\mathsf{Pr}[|S_y - E[S_y]| > \kappa] \leq 2e^{-2\kappa^2/y}.$$

For $t < O(\kappa^2)$, we have that this probability is negligibly small. Therefore with overwhelming probability, in the first run, there are more than t and less than $y - t$ queries that share the validity bit with the validity bit of the challenge. So there exists at least one spot that shares the validity bit with the challenge statement so that no external communication occurs. This spot is considered to be a useful rewinding spot. □

Now, let us analyze the running time of the meta-reduction. The proof is similar to that of Lemma 5, except for some small but non-trivial changes.

Lemma 11. *M has expected polynomial running time (in κ).*

Proof. We show that meta-reduction M has an expected polynomial running time. Most steps of the meta-reduction are efficient by construction. We have to analyze the running time of the rewinding phase, which is dominated by steps $2'.1.1.1$ – $2'.p.y.2$. We show that the expected number of jumps within these steps is polynomial. We define $E_{i,n}$ as *the event, that after sending a PCA query to B in state state_n, the meta-reduction receives a response w_i^* while B did not perform an external query.* By Lemma 9, it follows that B accepts an arbitrary

configuration with probability ϵ'. Let $((c_1, m_1), ..., (c_y, m_y))$ be the configuration that has been sent in the first run. Then, we know that this configuration is accepted by B, as otherwise the rewinding procedure would not have been started. Any individual statement (c_i, m_i) will be responded to with probability at least ϵ'. Assume that we load the internal state_n of B (for $n \in [y]$) in order to find the validity bit of w_i^* for $i \in [p]$. Then, the configuration will be of the following form $((c_1, m_1), ..., (c_n, m_n), (c_i', m_i'))$ where (c_i', m_i') is a randomization of the challenge statement (c_i^*, m_i^*). This configuration is distributed in the same way as the one from the ideal attacker A, except with probability δ. This is because the ideal attacker uses $\mathsf{RSample}$ to generate ciphertext/plaintext pairs. But $\mathsf{RSample}$ has success probability $1 - \delta$. By Lemma 10, we know that among the y queries, there exists a useful rewinding spot. Therefore, we have that $\mathsf{Pr}[E_{i,n}] \geq (\epsilon' - \delta)/y$. Of course, the same lower bound holds if we consider the event that for at least one of the possible rewinding spots, B will accept the configuration, i.e. $\mathsf{Pr}[E_i] = \mathsf{Pr}[E_{i,1} \vee ... \vee E_{i,y}] \geq (\epsilon' - \delta)/y$. Therefore, the running time of the meta-reduction for finding a validity bit for challenge s_i^* is $x_i = O(1/\mathsf{Pr}[E_i])$, which is polynomial. We repeat this argument p times, once for each challenge statement. Therefore, the expected number of jumps that are made to answer all challenge statements remains polynomial. This shows that the expected running time of the meta-reduction is polynomial. □

Lemma 12. *If M does not abort early on, it outputs validity bits $w_1^*, \ldots, w_p^*$. These validity bits are valid with a probability v that is non-negligibly better than $1/2^p$.*

Proof. By our construction, the meta-reduction M always outputs validity bits $w_1^*, ..., w_p^*$ when it does not abort. This is because, per iteration of rewinding, M either receives a validity bit or the reduction B has aborted the process. This does not guarantee the correctness of $w_1^*, ..., w_p^*$ yet, which we will show next.

Since M cannot recognize if the responses from the reduction are correct it can only hope that for *all* p challenges the computed validity bits are correct as well. This accounts for a success probability $((\epsilon' - \delta)/y)^p$ of M that is still non-negligible. So with non-negligible probability, the response of the reduction is correct as well. □

We apply the previous two lemmas to show that the reduction is not able to distinguish the ideal attacker and the meta-reduction.

Lemma 13. *The reduction B is not able to distinguish A and M with probability statistically close to one.*

Proof. We show that the reduction B is not able to distinguish A and M with a probability statistically close to one. The constructions of the ideal attacker A and meta-reduction M show that different behavior between A and M only occurs if M completes step $2'$ (so it did not abort), while receiving an invalid validity bit w_i for statement (c_i, m_i). We have already shown that for each configuration, $((c_1, m_1), ..., (c_y, m_y)) \in \mathcal{C} \times \mathcal{M}$, there is a rewinding spot (with overwhelming probability) for which the reduction B does not perform an external query to its tICA challenger before responding to it. The response of the reduction must must have been valid in the first run, as both A and M would have already aborted otherwise. As a query in the first run is distributed similarly to any query in the second run (except with probability δ), B cannot distinguish the behavior between A and M with a probability statistically close to 1. A final observation is that the reduction is not able to distinguish any behavior in the final output of A and M, as these are unique if both are valid. □

Finally, we show that the meta-reduction makes at most t queries to the ICA challenger.

Lemma 14. *Meta-reduction M will make at most t queries to the ICA challenger.*

Proof. We observe that the view of B, after rewinding has finished, is $((c_1, m_1), ..., (c_y, m_y))$. This tuple is distributed in the same way as the one that B has received from the ideal attacker A. Also M has never made more than t queries to the challenger in Step 2 overall. □

To wrap up, Lemmas 9 – 14 show that the meta-reduction simulates the ideal attacker, where reduction B is only able to distinguish this behavior with probability non-negligibly close to 1. In Section B.7, we present some applications of the impossibility result.

B.7 Applications of Theorem 4

We discuss some interesting applications of Theorem 4. We can apply the theorem to semi-homomorphic PKE and obtain the following corollaries on IND-PCA1 security.

Corollary 4. *Let κ be the security parameter. Let p, the number of challenge statements that the attacker must solve in the IND-PCA1 security game, be constant. Then, there exists no simple PPT reduction B that reduces the IND-PCA1 security of a certified semi-homomorphic PKE with $2t + 2\kappa$ adaptive PCA queries to the security of any certified semi-homomorphic PKE with t PCA queries in the IND-PCA1 game, for $t < O(\kappa^2)$.*

We can more specifically apply our result to ElGamal PKE (see Figure 5).

Corollary 5. *Let κ be the security parameter. Let p, the number of challenge statements that the attacker must solve in the IND-PCA1 security game, be constant. Then, there exists no simple PPT reduction B that reduces the IND-PCA1 security of ElGamal PKE with $2t + 2\kappa$ adaptive PCA queries to the security of any t-interactive complexity assumption, for $t < O(\kappa^2)$.*

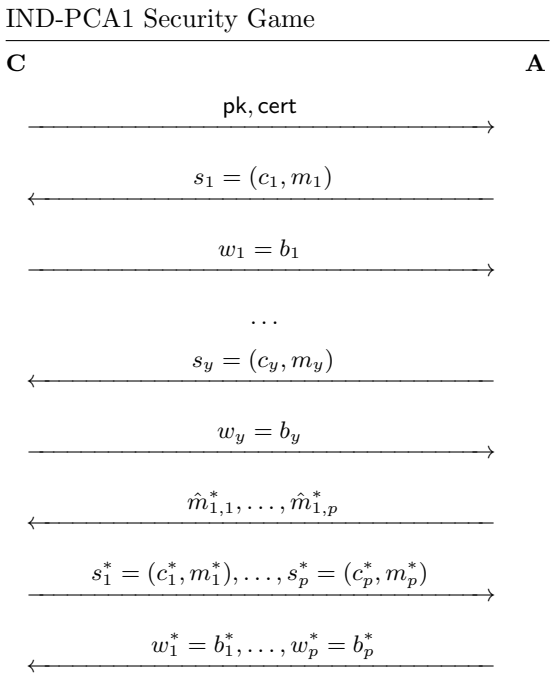

Figure 4: The IND-PCA1 security game between challenger **C** and attacker **A**. On initialization, a public key pk together with a certificate cert of a SHPKE are sent to the attacker. The attacker may query y PCA queries of the form $s_i = (c_i, m_i)$ for $i \in [y]$. The challenger will respond with $w_i = 1$ if s_i is a valid ciphertext/plaintext pair for the SHPKE. Otherwise, **C** responds with bit $w_i = 0$. After t queries, the attacker is given p challenge statements $s^*_1 = (c^*_1, m^*_1), \ldots, s^*_p = (c^*_p, m^*_p)$, for which it has to determine the corresponding validity bits $w^*_1, ..., w^*_p$.

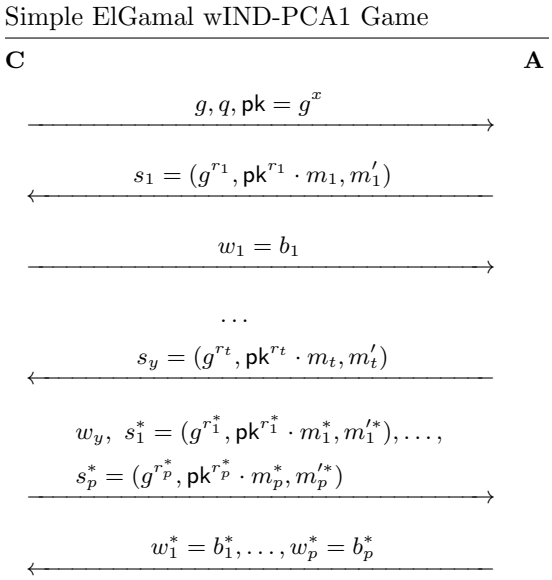

Figure 5: The wIND-PCA1 security game for ElGamal between challenger **C** and attacker **A**. On initialization, a public key pk together with a certificate cert of a SHPKE are sent to the attacker. The attacker may query y PCA queries of the form $s_i = (g^{r_i}, \mathsf{pk}^{r_i} \cdot m_i, m'_i)$ for $i \in [y]$. The challenger will respond with $w_i = 1$ if s_i is a valid ciphertext/plaintext pair for the SHPKE. Otherwise, **C** responds with bit $w_i = 0$. After y queries, the attacker is given p challenge statements $s_1^* = (g^{r_1^*}, \mathsf{pk}^{r_1^*} \cdot m_1^*, m_1'^*), \ldots, s_p^* = (g^{r_p^*}, \mathsf{pk}^{r_p^*} \cdot m_p^*, m_p'^*)$, for which it has to determine the corresponding validity bits $w_1^*, ..., w_p^*$.

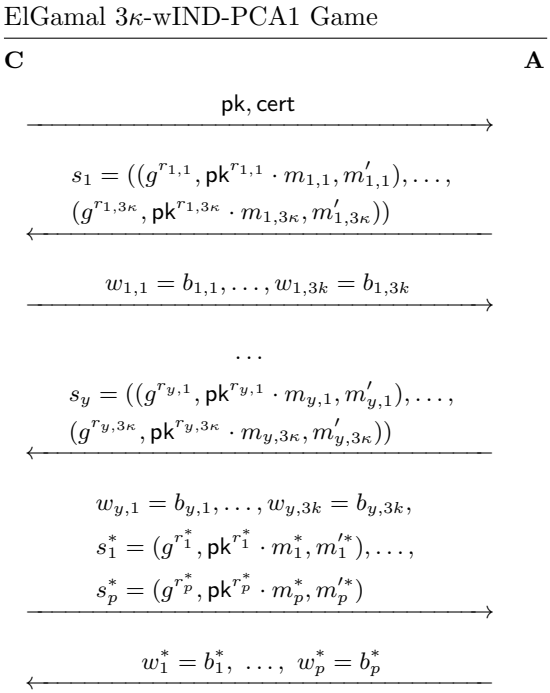

Figure 6: The 3κ-wIND-PCA1 security game for ElGamal between challenger **C** and attacker **A**. On initialization, a public key pk together with a certificate cert of a SHPKE are sent to the attacker. The attacker may query y 3κ-PCA queries of the form $s_i = ((g^{r_{i,1}}, \mathsf{pk}^{r_{i,1}} \cdot m_{i,1}, m'_{i,1}), ..., (g^{r_{i,3\kappa}}, \mathsf{pk}^{r_{i,3\kappa}} \cdot m_{i,3\kappa}, m'_{i,3\kappa}))$ for $i \in [y]$. The challenger responds with $w_{i,j} \in \{0,1\}$ for each tuple $(g^{r_{i,j}}, \mathsf{pk}^{r_{i,j}} \cdot m_{i,j}, m'_{i,j})$. After y queries, the attacker is given p challenge statements of the form $s_1^* = (g^{r_1^*}, \mathsf{pk}^{r_1^*} \cdot m_1^*, m_1'^*), \ldots, s_p^* = (g^{r_p^*}, \mathsf{pk}^{r_p^*} \cdot m_p^*, m_p'^*)$, for which it has to determine the corresponding validity bits $w_1^*, ..., w_p^*$.